Microservice Autoscaling With Fine-Grained DVFS in Dynamic Multi-Edge Collaboration via Deep Reinforcement Learning

Bo Xu, Ruyao Zou, Jiangtian Nie, Zhiheng Yao, Yi Hu, Hao Wen, Menglan Hu, and Kai Peng, *Member, IEEE*

***Abstract*—Microservice architecture has become the mainstream approach for low-latency and robust edge scenarios. In practice, service demands exhibit high temporal variability and concurrency, making efficient autoscaling significantly critical. However, existing research on autoscaling adopts coarse-grained service models that treat instance adjustment and request routing in isolation. This simplification neglects the intricate interdependencies among services, thus undermining the efficacy of autoscaling. In addition, many current autoscaling models do not simultaneously account for autoscaling and dynamic voltage and frequency scaling (DVFS), making it difficult to achieve an effective balance between performance and energy consumption. To overcome these challenges, this paper proposes an autoscaling algorithm that synergistically combines deep reinforcement learning with DVFS. First, we build a multi-instance performance model grounded in queuing theory, formulating microservice autoscaling as an optimization that minimizes both latency and energy consumption. Second, we present the Reinforcement Learning and DVFS Joint Optimization Autoscaling Algorithm (RLDA), which observes system state changes and scales microservice instances in real time while jointly optimizing DVFS. Third, to balance latency and energy consumption, we design the Latency-Constrained Greedy Frequency Scaling Algorithm (LGFS), which adjusts server operating voltage based on deployment scheme. Extensive simulation results demonstrate that under the Twitter traffic pattern, our algorithm achieves comprehensive performance improvements in energy consumption and latency of 37%, 25%, and 29% compared to DRL, ProScale, and PPA, respectively. Under the World Cup traffic pattern, the gains are 33%, 28%, and 23%, respectively.**

***Index Terms*—Edge computing, microservices, autoscaling, deep reinforcement learning.**

I. Introduction

EDGE computing, as an emerging paradigm, can significantly reduce service latency and meet the quality-of-service (QoS) requirements of latency-sensitive Internet applications [1], [2], [3]. To effectively address the complexity inherent in edge environments, leading Internet companies including Google and Alibaba have adopted microservice architecture to build applications [2], [4], [5]. In this architecture, monolithic applications are decoupled into multiple modules, each organized around a specific business capability and collaborates with others to provide complete functionality to Internet applications [6], [7].

In edge computing environments, user requests exhibit high temporal variability over time, necessitating continuous readjustment of microservice instances [8], [9]. The challenge of the readjustment is further intensified in real-world scenarios, where Internet applications are computationally intensive and highly energy-demanding, yet edge servers suffer from constrained computing resources and energy supply [10]. Therefore, it is imperative to develop autoscaling schemes that jointly optimize both latency and energy consumption [11]. However, existing research on autoscaling has overlooked the trade-off between latency and energy consumption, leaving numerous related challenges largely unresolved.

First, an effective autoscaling strategy requires the joint consideration of instance adjustment and request routing. Due to the complex data communication and invocation dependencies, instance adjustment and request routing exhibit a tightly coupled and interdependent relationship [12]. Ambiguity in routing policies undermines the ability to assess the efficacy of the proposed instance adjustment scheme, while the absence of a concrete plan for scaling instances leaves routing decisions without a reliable foundation [13], [14]. Therefore, to improve system performance, it is essential to jointly address the microservice instance adjustment and request routing within a service mesh architecture. However, most existing studies treat instance adjustment and request routing as separate problems [15], [16]. Such decoupled optimization approaches neglect the explicit interactions between these two components, thereby failing to achieve efficient autoscaling.

Second, intelligent autoscaling relies on multi-instance modeling and fine-grained delay analysis. Given the limited

Received 3 April 2026; accepted 30 April 2026. Date of publication 11 May 2026; date of current version 17 June 2026. This work was supported in part by the Regional Innovation and Development Joint Fund of the National Natural Science Foundation of China under Grant U25A20435 and in part by the Key Research and Development Program of Hubei Province under Grant 2025BAB051. Recommended for acceptance by Prof. Yao Sun. *(Corresponding author: Yi Hu.)*

Bo Xu, Ruyao Zou, Zhiheng Yao, Yi Hu, Hao Wen, Menglan Hu, and Kai Peng are with the Hubei Key Laboratory of Internet of Intelligence, School of Electronic Information and Communications, Huazhong University of Science and Technology, Wuhan 430074, China (e-mail: xubosite@hust.edu.cn; ruyaozou@hust.edu.cn; zhihengyao@hust.edu.cn; hust_huyi@hust.edu.cn; whhust@hust.edu.cn; humenglan@hust.edu.cn; pkhust@hust.edu.cn).

Jiangtian Nie is with the Department of Computing Science, University of Aberdeen, AB24 3FX Aberdeen, U.K. (e-mail: jiangtian.nie@abdn.ac.uk).

Digital Object Identifier 10.1109/TNSE.2026.3691933

2327-4697 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on September 16,2026 at 05:06:15 UTC from IEEE Xplore. Restrictions apply.

resources of edge computing and the explosive growth of end-user requests, deploying multi-instances is essential to sustain high throughput [17]. Nevertheless, many studies model microservices as single-instance entities, neglecting the multi-instance reality that prevails in production [18], [19]. Although this simplification facilitates analysis, it struggles to handle high traffic and fails to address intricate interdependencies among services. Multi-instance autoscaling offers greater granularity and practical relevance, but it requires joint consideration of the number and deployment locations of microservice instances, necessitating more refined end-to-end delay analysis to support precise decisions. This dilemma is further exacerbated in large-scale dynamic scenarios where computational resources are constrained and delay-sensitive. Therefore, designing a multi-instance model that enables fine-grained autoscaling through accurate delay analysis is critical for jointly optimizing latency and energy consumption.

Third, the energy consumption in autoscaling cannot be overlooked. In edge computing scenarios, energy efficiency must be carefully managed to accommodate stringent resource constraints [20], [21]. However, previous studies primarily focused on QoS metrics such as latency and computational costs, failing to achieve a balance between power consumption and performance. Some studies attempted to reduce energy consumption by decreasing the number of servers and microservice instances [22], [23]. Yet energy consumption is closely coupled with CPU frequency, and indiscriminate frequency reduction can slow down processing speeds and increase latency [24]. Under such conditions, we present a fine-grained dynamic voltage and frequency scaling (DVFS) strategy that is jointly optimized with autoscaling. This strategy tracks instantaneous load changes in real-time, aiming to minimize energy consumption while satisfying the maximum latency constraints, thereby achieving a balance between energy consumption and latency.

Motivated by the above needs and research gaps, this paper focuses on a microservice autoscaling algorithm that optimizes energy consumption and latency in edge computing, aiming to achieve low-latency, energy-efficient placement of microservice applications within resource-constrained and dynamic edge environments. To achieve this goal, several challenges must be addressed. Firstly, implementing autoscaling of instances to accommodate changing user requests is a significant problem. Second, this difficulty is further intensified when considering multi-instance modeling for large-scale, intricate microservice instances. Finally, achieving low latency inevitably leads to a significant increase in energy consumption, while reducing energy usage inevitably extends service latency. This inherent trade-off between latency and energy consumption is the core challenge in addressing the complex issues discussed in this paper.

To address these challenges, this paper first establishes a queuing-theoretic dependency model that precisely captures the intricate invocation relationships among microservices. Building upon this foundation, we devise an autoscaling algorithm integrates reinforcement learning with DVFS. Tailored for time-varying edge workloads, the scheme employs deep reinforcement learning to elastically scale microservice instances while, in tandem, adapting CPU frequencies to each server's processing capacity and instantaneous delay budget, boosting service capacity and curtailing system-wide energy expenditure. The key contributions of this work are threefold:

- We formulate the microservice autoscaling problem as a multi-objective optimization, whose goal is to minimize both latency and energy while satisfying all operational constraints. Based on queuing-network theory, we construct a performance model that accurately captures the interdependencies among microservices and across the entire request-processing workflow.
- We propose the Reinforcement Learning and DVFS Joint Optimization autoscaling Algorithm (RLDA). Tailored to the time-varying nature of edge environments, RLDA employs a deep reinforcement-learning agent to perceive state changes and autonomously adjust the scaling decisions of microservice instances.
- We devise a Latency-Constrained Greedy Frequency Scaling Algorithm (LGFS), which employs a greedy optimization strategy to independently optimize each edge node, efficiently adjusting server voltage to balance latency and energy consumption.
- Experiments demonstrate that the algorithm achieves comprehensive optimization of latency and energy consumption across various dynamic traffic patterns, exhibiting greater adaptability and robustness compared to traditional strategies.

The remainder of this paper is structured as follows. Section II offers a concise review of existing literature on the subject. The problem model under discussion is formulated in Section III. The proposed algorithms are detailed in Section IV. In Section V, the results of the algorithm performance evaluation are presented. Finally, Section VI concludes the paper.

II. Related Work

In recent years, significant attention has been devoted to delivering high-quality and reliable services for dynamic user requests based on edge computing with microservice architectures [17], [25]. Existing studies were broadly classified into three main directions: dynamic performance optimization, latency optimization, and energy consumption optimization. Specifically, dynamic performance optimization improves system stability through autoscaling. To reduce end-to-end response time, latency optimization employs intelligent joint optimization of service deployment and request routing. Meanwhile, energy consumption optimization leverages DVFS to strike a balance between energy efficiency and performance. Table I summarizes the key differences between our work and existing related studies, thereby clearly highlighting the novelty and contributions of this paper. In addition, a detailed review of representative work in these areas is provided below.

A. Dynamic Performance Optimization

In microservice systems targeting edge computing environments, service requests exhibit high dynamism, making static

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on September 16,2026 at 05:06:15 UTC from IEEE Xplore. Restrictions apply.

TABLE I
COMPARISON OF RELATED WORK

Ref.	Autoscaling	Joint Optimization of Deployment and Routing	DVFS	Optimization objective	Solution
Cheng et al. [16]	✓	✓	×	Delay	Heuristic
Li et al. [26]	✓	×	×	Delay	Heuristic
Dimolitsas et al. [28]	✓	×	×	Cost and energy consumption	Heuristic
Ding et al. [35]	✓	×	×	QoS and cost	DRL
Peng et al. [36]	✓	✓	×	Delay and energy consumption	DRL
Fu et al. [22]	×	×	×	QoS and cost	DRL
Hu et al. [42]	×	×	×	Delay	Deep learning
He et al. [45]	×	×	×	Delay and QoS	DRL
Liang et al. [47]	×	✓	×	Energy consumption	Heuristic
Lai et al. [48]	✓	✓	×	Cost	Heuristic
Premsankar et al. [57]	×	✓	×	Cost and energy consumption	Heuristic
Sun et al. [59]	×	✓	×	Cost and energy consumption	Heuristic
Varasteh et al. [60]	×	✓	×	Energy consumption	Heuristic
Zhang et al. [61]	×	×	✓	Delay and energy consumption	DRL
Wang et al. [63]	×	✓	✓	Delay and energy consumption	DRL
Chen et al. [66]	×	×	✓	QoS	DRL
Ours	✓	✓	✓	Delay and energy consumption	DRL

deployment approaches ineffective and resulting in both resource inefficiency and elevated latency.

Some studies formulated the autoscaling problem as an optimization task and addressed it using mathematical models and heuristic algorithms. Cheng et al. [16] employed the Lyapunov optimization framework to decompose the complex long-term autoscaling problem into a sequence of subproblems and applied a signomial geometric programming (SGP)-based algorithm to obtain near-optimal solutions for each NP-hard subproblem. Li et al. [26] addressed the heterogeneity of edge systems by jointly considering microservice autoscaling and placement, proposing an adaptive particle swarm optimization algorithm. μOpt [27] computed optimal configurations that maximize performance while minimizing allocated resources by solving a nonlinear optimization problem embedding a Layered Queueing Network (LQN) model. Dimolitsas et al. [28] applied the Analytic Hierarchy Process (AHP) to balance multiple objectives and made cluster-level scaling decisions. Although these approaches often provided performance guaranties, they were typically sensitive to the accuracy of model assumptions and parameter settings.

Further advances led to hybrid autoscaling methods that integrated both reactive and proactive mechanisms, with a core focus on workload forecasting. Pramesti et al. [29] employed a machine learning model to develop an autoscaler for containers based on the prediction of response time. Al Qassem et al. [30] introduced a novel proactive autoscaling approach featuring a two-stage machine learning pipeline based on Random Forest (RF) to predict future demands for CPU and memory utilization of microservices. Li et al. [31] proposed DCScaler, a distributed cooperative autoscaler that leveraged spatiotemporal prediction capabilities to proactively adjust resource allocation. Chen et al. [32] proposed a hybrid autoscaling policy that combines proactive and reactive mechanisms, which predicts future traffic loads and the corresponding startup times of microservices to proactively adjust computing resources and optimize resource scheduling. Cheng et al. [16] leveraged large-scale microservice trace data to proactively forecast the workload of each microservice in each time slot and make scaling decisions accordingly. Although effective in handling traffic surges, such predictive methods relied heavily on the accuracy of their forecasts.

Recently, approaches based on deep learning and reinforcement learning attracted significant attention. These data-driven methods learned scaling policies either from historical data or through interaction with the environment. Khaleq et al. [33] deployed reinforcement learning agents that autonomously scaled resources according to demand and QoS targets. Nayir et al. [34] presented a multidimensional autoscaling framework using Deep Q-Networks (DQN). Ding et al. [35] introduced a joint scaling strategy based on Long Short-Term Memory-Deep Reinforcement Learning(LSTM-DRL) for heterogeneous containers in collaborative power cloud-edge environments. Peng et al. [36] developed a cluster-based autoscaling algorithm to tackle resource heterogeneity and long-term optimization challenges. DeepScaler [37] achieved end-to-end microservice autoscaling through an attention-based graph neural network (GNN) model combined with an adaptive graph learning method. Building on this, Fang et al. [38] integrated GNNs with an information aggregation mechanism to enable efficient scaling decisions in response to rapidly varying user request loads. These DRL-based methods adopt a model-free and interaction-driven strategy. By employing agents that directly derive optimal autoscaling policies through trial-and-error learning, this approach maximizes long-term cumulative rewards. Consequently, DRL agents can adaptively manage complex nonlinear system behaviors and

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on September 16,2026 at 05:06:15 UTC from IEEE Xplore. Restrictions apply.

abrupt workload surges without relying on precise mathematical formulations or accurate workload predictions. This ensures superior robustness and real-time responsiveness in highly volatile multi-edge collaboration scenarios.

Although the aforementioned autoscaling studies improved system stability and QoS to some extent in dynamic environments, most of them neglected the joint optimization of autoscaling and request routing. Such decoupled approaches failed to effectively handle complex invocation dependencies among microservices. Moreover, a common limitation across these works is the lack of a fine-grained, multi-instance service model, which further exacerbates system instability and hinders the realization of intelligent autoscaling.

B. Latency Optimization

Latency optimization has long been a central focus in microservice orchestration. Most prior studies treated deployment and routing as two independent modules to be optimized separately, while only a few recognized their tightly coupled relationship and pursued joint optimization to further reduce end-to-end latency.

1) Separate Optimization of Deployment and Routing: A line of research focused on optimizing instance deployment. Fu et al. [22] investigated resource contention among microservices in edge–cloud scenarios. They first applied a mapping algorithm to partition and deploy microservices, then performed real-time workload monitoring to enable adaptive resource adjustment and dynamic migration, thereby meeting QoS objectives. He et al. [39] addressed microservice systems with complex dependencies by formulating the service placement problem as a fractional polynomial programming model and proposed an efficient greedy-based solution. Li et al. [40] introduced an edge service deployment strategy based on an improved fast non-dominated sorting genetic algorithm (GA-MSP), which minimized transmission latency and balanced load through optimized microservice deployment. Lv et al. [41] jointly considered the frequency of interaction between microservices and load balancing, proposing a deep Q-learning algorithm with reward sharing. This algorithm effectively reduced response time, achieved resource load balance, and enabled autoscaling. These deployment-only approaches primarily targeted the reduction of processing delay.

Another stream of work emphasized request routing to mitigate latency. Hu et al. [42] proposed an end-to-end optimization framework based on graph neural networks to address intelligent routing in microservice systems. Desai et al. [43] designed three routing algorithms that can be deployed in software-defined networks (SDN) to reduce latency. Liu et al. [44] introduced an innovative Fault-Tolerant Traffic Routing (FTTR) mechanism that integrated Message Passing Neural Networks (MPNNs) with DQN to capture inter-link dependencies and make accurate routing decisions. He et al. [45] developed a real-time, distributed learning approach capable of adapting to highly volatile network conditions and enabling hop-by-hop routing. Casas-Velasco et al. [46] overcame the limitations of traditional reinforcement learning methods by proposing an intelligent routing approach based on Deep Reinforcement Learning and Software-Defined Networking, termed DRSIR. These routing-centric studies mainly aimed to improve propagation delay along end-to-end paths.

2) Joint Optimization of Deployment and Routing: Recognizing the interdependence between deployment and routing, several recent efforts adopted a joint perspective. Liang et al. [47] modeled the joint placement of Virtual Network Functions (VNFs) and traffic routing as an integer nonlinear programming problem and proposed an online approximation policy with a constant approximation ratio that efficiently solved each single-timeslot subproblem in polynomial time. Lai et al. [48] presented a joint optimization strategy for microservice autoscaling and request routing, and designed an innovative multistage heuristic algorithm (MFDS-FPR) that adapted to dynamic user request traffic while minimizing deployment cost. Xiong et al. [49] developed a Transformer-enhanced Twin Delayed Deep Deterministic Policy Gradient (TETD3MD) algorithm for microservice deployment together with an A*-based, queue-aware multi-chain scheduler to jointly optimize deployment and multi-chain request routing. Peng et al. [50] focused on joint optimization in multi-edge-cloud scenarios and proposed a deep reinforcement learning-based algorithm to minimize average response latency. Building on this, they further considered the fine-grained hybrid orchestration of AI services and microservices and utilized deep reinforcement learning to jointly optimize service deployment and request routing [51], [52], [53]. Hu et al. [14] investigated dynamic service mesh orchestration for tree-like microservice graphs in cloud environments using a probabilistic routing mechanism. These joint approaches simultaneously addressed both processing and propagation delays, thus achieving more comprehensive latency reduction. However, most of them lacked fine-grained modeling and detailed latency decomposition, leaving considerable room for improvement.

C. Energy Consumption Optimization

Given the limited energy supply of edge devices, algorithm design must incorporate energy consumption considerations to enable efficient microservice orchestration.

1) Energy Efficiency Through Resource Scheduling: Bi et al. [23] addressed the task-offloading problem in hybrid edge systems by proposing a genetic learning-based particle swarm optimization algorithm, explicitly formulated to minimize the total energy consumption of the system. Gu et al. [54] categorized energy as green or brown, arguing that edge computing should prioritize green energy to reduce costs. Based on this categorization, they introduced a Deep Reinforcement Learning (DRL)-based task offloading algorithm that comprehensively considered energy availability and user demands. Teng et al. [55] designed an integrated algorithm for both task offloading and service deployment, tailored for tasks with sequential dependencies and multiple constraints. Rao et al. [56] proposed a microservice scheduling strategy for containerized clusters that took into account communication frequency and resource usage while focusing on energy efficiency and low-latency service delivery. Premsankar et al. [57] formulated the energy-efficient

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on September 16,2026 at 05:06:15 UTC from IEEE Xplore. Restrictions apply.

placement of AI application services as a multi-period optimization problem and minimized overall energy consumption and latency through joint optimization of service placement and request scheduling. Wang et al. [58] proposed a joint energy and computational workload management framework aimed at establishing a sustainable data center paradigm integrated with distributed renewable energy sources. Sun et al. [59] dynamically adjusted the delayed shutdown duration of servers based on the actual utilization of each device in every time slot to reduce energy consumption. Varasteh et al. [60] optimized energy efficiency by minimizing the number of powered-on physical machines (PMs). However, these studies were largely confined to single-task offloading paradigms or specific application scenarios and did not account for the tight coupling between energy consumption and CPU frequency, imposing great difficulty to achieve a balanced trade-off between performance and energy efficiency.

2) Energy Efficiency Through DVFS: Recent work also explored hardware-level techniques to further reduce energy consumption. Zhang et al. [61] introduced a deep neural network (DNN) inference framework supporting DVFS for edge intelligence applications. This framework optimized autoscaling on edge devices and task allocation across cloud and edge resources, effectively reducing energy consumption and inference latency. Khaleel et al. [62] introduced a multi-objective ant colony optimization algorithm for service deployment in edge-cloud environments and further applied DVFS to enhance energy savings. Wang et al. [63] jointly optimized static microservice deployment and request routing and designed a dynamic energy-saving algorithm based on DVFS to adapt to fluctuating request loads. Liu [64] and Yeganeh-Khaksar [65] proposed an enhanced reinforcement learning-based DVFS technique to reduce power consumption. Several studies [66], [67], [68], [69], [70] achieved multi-objective optimization by integrating Deep Reinforcement Learning with DVFS. These efforts made significant progress in energy-efficient edge computing and offered diverse approaches to reduce energy consumption in edge systems. Nevertheless, most existing studies failed to integrate energy consumption and latency into a unified optimization framework, limiting their effectiveness in large-scale, dynamic microservice orchestration scenarios.

To our knowledge, no existing study integrated autoscaling, joint deployment and routing optimization, and DVFS to achieve coordinated optimization of dynamic performance, latency, and energy consumption.

III. Problem Formulation

In this section, we begin by defining the notation that will be employed throughout this paper. We then provide a concise introduction to the problem scenario and elaborate on the problem model. A summary of the key symbols is presented in Table II for ease of reference.

TABLE II
Notation and Terminology

Notation	Definition
Z	Set of edge servers
E	Set of wireless links between servers
B_{z_i,z_j}	Link bandwidth between servers Z_i and Z_j
M, m	Set of microservices, microservice
$\mu_m^z(t)$	Average service rate of microservice m on server z at time t
$N_m^z(t)$	Instances of microservice m on server z at time t
D_m^f	Data volume for request f to microservice m
d_f^{max}	Maximum tolerable delay for each service request
$R_{f,m}^z$	Routing probability of microservice m in request f to server z
$F(t)$	Set of service requests arriving at the system at time t
$\lambda_f(t)$	Average arrival rate of each request $f \in F(t)$
$\tau_{f,m}^{z,z_j}$	Transmission time for the data required by microservice m in request f from edge server Z_i to edge server Z_j
$T_f^t(t)$	Expected delay for data transmission between different edge servers during the processing of request f at time t
$\lambda_m^z(t)$	Total arrival rate to microservice m on server z at time t
$\rho_m^z(t)$	Service intensity of microservice m on server z at time t
$T_{m,z}^q(t)$	Average queuing delay of service requests at time t
$T_f^q(t)$	Average queuing delay of request f across all paths at time t
$T_f^c(t)$	Sum of average processing delays in request f
$T(t)$	Normalized average delay at time t
P_z^{idle}	Static power consumption of edge server
P_{dyn}	Dynamic power consumption of edge server
$U_z(t)$	CPU utilization rate of edge server at time t
P_z^{max}	Power consumption of edge server z at maximum voltage load

A. System Models

To facilitate the description of geographically dispersed edge systems, this paper models them using a weighted undirected graph $G = (Z, E)$. In this model, Z represents the collection of edge devices, while E represents the set of wireless links between servers. Edge servers possess computing and storage capabilities similar to those of cloud computing centers and can deploy microservice instances via containerization to receive and process requests from user devices. Early research [71] identified CPU resource limitations as a key bottleneck affecting server performance. Therefore, resource capacity at each edge server is represented by number of CPU cores C_z provisioned on its server, where each core is capable of launching a container to host a microservice instance. Furthermore, edge servers can communicate with each other through wireless links, allowing them to forward requests to adjacent servers and collaboratively fulfill user requests. For the link between servers Z_i and Z_j, the bandwidth is denoted as $B_{(Z_i,Z_j)}$.

B. Dynamic Requests and Scaling of Microservices

Adopting the concept of microservices, this paper modularizes edge system functions into a series of discrete microservices. Let M denote the set of microservices, where each $m \in M$ represents a distinct type of microservice. These microservices are instantiated on edge servers and assigned to

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on September 16,2026 at 05:06:15 UTC from IEEE Xplore. Restrictions apply.

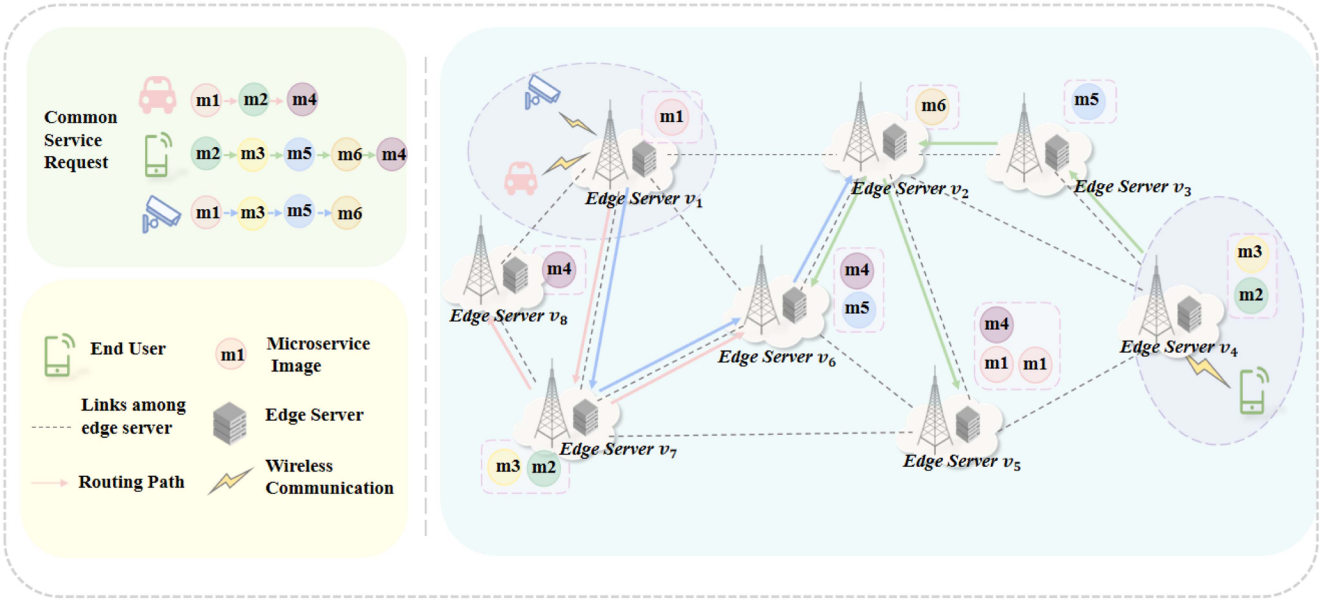


Fig. 1. Deployment Diagram of Edge System Microservices.

different functional roles within the system. The service rate μ_m, defined as the inverse of the average service time, characterizes the processing capacity of a microservice per unit time. Fig. 1 illustrates the deployment diagram of microservices in the edge system. Each type of microservice is allowed to have multiple identical instances, which operate independently within separate containers without resource conflicts. This multi-instance mechanism facilitates horizontal scaling by distributing incoming request loads across parallel processing units, thereby effectively mitigating single-point congestion, significantly reducing queuing delays, and enhancing overall system throughput and service reliability under dynamic traffic fluctuations.

Define the deployment decision variable N_m^z to represent the number of microservice m instances deployed on server z. Given the limited resources at edge servers, the total number of deployed microservice instances on each server must not exceed its available CPU cores:

$$\sum_{m \in M} N_m^z \leq C_z, \forall z \in Z \tag{1}$$

This paper employs F to represent the set of service requests types. Each request f can be fulfilled by a sequence of microservices working together. Therefore, a microservice chain $M_f = \{m_1, m_2, m_3, \ldots, m_n\}$ can be used to describe the workflow of services in edge computing, where $m_1, m_2, m_3, \ldots, m_n$ represents the set of microservices. To complete a task, a service request must pass through these microservices in a specified order. For example, as shown in Fig. 1, a request from the surveillance system must go through four microservices m_1, m_3, m_5, and m_6 to obtain the final result. Different requests may reuse some basic microservices. As illustrated in Fig. 1, both the vehicular network request and the surveillance request invoke the microservice m_1.

During the processing of microservices, the volume of request data may vary. Therefore, this document uses D_f^m to represent the amount of data that must be transferred to the microservice m for processing within a request f. Furthermore, considering that different application services have varying requirements for real-time performance, each service request has a different maximum tolerable delay $d_f^{\max}$.

In practical application scenarios, the request traffic received by the system is not constant but exhibits significant time-varying characteristics. To accurately capture the dynamics and volatility of request traffic, we discretize time into a series of equal length time slots $t \in T$ and periodically adjust the microservice deployment scheme per slot to adapt to dynamic changes in the edge environment.

Let the set of service requests arriving at the system in a time slot be denoted as $F(t)$. For each request $f \in F(t)$, the average arrival rate is $\lambda_f(t)$. It is assumed that the request arrival process follows a time-varying Poisson distribution, that is, within each time slot, the request arrival rate follows a Poisson distribution whose parameter $\lambda_f(t)$ varies with t. This piecewise stationary assumption can not only fit the dynamic characteristics of the actual request flow well but also simplify the complexity of modeling and analysis.

To address the dynamic nature of requests, the system must possess the capability of autoscaling, which involves dynamically adjusting the deployment of microservice instances in response to changes in request traffic. For example, when there is a sudden increase in user requests, the system needs to dynamically expand the number of service instances to distribute the load, thus maintaining service quality and preventing request accumulation and response timeouts. Conversely, when the request volume decreases, the system should promptly reduce the number of instances to lower resource consumption and avoid energy waste. To describe this process, we introduce the instance

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on September 16,2026 at 05:06:15 UTC from IEEE Xplore. Restrictions apply.

scaling decision variable:

$$\Delta N_m^z(t) = N_m^z(t) - N_m^z(t-1) \tag{2}$$

where $N_m^z(t)$ denotes the number of instances of the microservice m deployed on the server z in time interval t. The variable $\Delta N_m^z(t)$ represents the change in the number of instances between adjacent time slots. If $\Delta N_m^z(t) > 0$, it indicates that an instance scaling-up operation is performed for the microservice on server z in time slot t; if $\Delta N_m^z(t) < 0$, it signifies that a scaling-down operation is executed; and if $\Delta N_m^z(t) = 0$, it means that the current instance configuration remains unchanged.

C. Request Routing Models

In real-world edge computing deployments, the distribution of microservice instances is highly dynamic and heterogeneous. Multiple instances of the same microservice may exist, and these instances can be located on the same edge server or dispersed across different geographical locations. This characteristic makes the routing of service requests typically complex and difficult to enumerate exhaustively.

To effectively model and optimize this complex routing behavior, this section introduces the concept of probabilistic routing. Define the probabilistic routing variable $R_{f,m}^z \in [0,1]$, which represents the probability that a request f that requires processing by a microservice m is forwarded to the edge server z and handled by the corresponding microservice instance at that server. To ensure the validity and effectiveness of the routing model, the probabilistic routing variables must satisfy the following constraints:

$$\sum_{z \in Z} R_{f,m}^z = 1, \forall f \in F, \forall m \in M \tag{3}$$

which ensures that the sum of routing probabilities for each request at each microservice stage is 1, thus preventing traffic loss or duplicate delivery.

$$R_{f,m}^z > 0 \Rightarrow N_m^z \geq 1 \tag{4}$$

which enforces feasibility by restricting routing decisions to servers where the corresponding microservice is actually deployed.

The determination of probabilistic routing directly impacts system load balancing and end-to-end latency, serving as a critical component in evaluating the performance of microservice deployment. Since the microservice architecture typically employs a stateless design, instances do not store session data for specific users, and any instance of the same microservice can handle requests from different users. Therefore, this paper adopts a probabilistic routing calculation strategy based on the number of deployed instances to achieve load balance and simplify the analysis. Specifically, the probability $R_{f,m}^z$ that a request f is routed to server z for microservice m can be calculated using the following formula:

$$R_{f,m}^z = \frac{N_m^z}{\sum_{z_i \in Z} N_m^{z_i}} \tag{5}$$

where N_m^z represents the number of instances of microservice m deployed on server z, and $\sum_{z_i \in Z} N_m^{z_i}$ is the total number of instances for microservice m in the system. This formula indicates that servers with stronger service capabilities will handle a larger proportion of request traffic.

D. Frequency Models Based on DVFS

Modern edge servers widely support DVFS technology, which dynamically adjusts the processor's working voltage and frequency to reduce energy consumption while meeting performance requirements. We assume that each server z in a heterogeneous edge server cluster supports K discrete performance states (P-states), denoted as the set $(P_z = \{p_1, p_2, \ldots, p_K\}$. Each p_k corresponds to a voltage-frequency pair (V_k, f_k), where V_k is the voltage and f_k the corresponding clock frequency. Here, the sequence $f_1 < f_2 < \ldots < f_K$ represents increasing frequency levels, reflecting the server's transition from low-power to high-performance operating states.

For the ease of modeling, we introduce a 0-1 decision variable $x_{z,k}(t)$, which indicates whether server z chooses performance state p_k in time slot t. The operating frequency of server z in time slot t can then be calculated using the following equation:

$$f_z(t) = \sum_{k=1}^{K} x_{z,k}(t) f_k \tag{6}$$

E. Multi-Objective Optimization Models

This study aims to jointly minimize system latency and energy consumption by optimizing both the number of microservice instances deployed in each time slot and the configuration of server hardware resources. The specific optimization objectives are as follows:

1) Request End-to-End Delay: In edge computing servers, the CPU's processing capacity is not only influenced by fixed physical parameters, but also exhibits a strong correlation with the operating frequency f, which affects the overall response delay of the edge network to user requests. According to DVFS theory, $\mu_m^z(t)$ is defined as the average service rate of microservice m on server z at time t, and its calculation formula is as follows:

$$\mu_m^z(t) = \mu_m \xi_z \frac{f_z(t)}{f_{\text{base}}} \tag{7}$$

where μ_m is the service rate of microservice m at the base frequency, f_{base} represents the base frequency, and $\xi_z \in (0,1]$ is an efficiency coefficient related to the server's chip manufacturing process and architecture. For simplification in this paper, it is assumed to be 1.

After a request arrives at the edge system, it traverses the corresponding microservices based on transition probabilities to complete the service. Thus, the process of accepting service for a request can be divided into two parts: locating the appropriate microservice according to the routing strategy, and accepting processing by the microservice.

During the process of locating the corresponding microservice based on the routing strategy, it may sometimes be necessary to forward the request to other servers, which adds extra transmission delay. The transmission time required to send the data

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on September 16,2026 at 05:06:15 UTC from IEEE Xplore. Restrictions apply.

needed by microservice m in request f from edge server Z_i to edge server Z_j can be calculated as follows:

$$T_{f,m}^{z_i,z_j} = \frac{D_m^f}{B_{z_i,z_j}} \tag{8}$$

since the internal communication delay within the server is generally negligible, if $i = j$, then $T_{f,m}^{z_i,z_j}$ is 0.

Considering the existence of transfer probabilities, the expected data transmission delay between different edge servers during the processing of request f can be represented as:

$$T_f^t(t) = \sum_{m,m' \in M_f} \sum_{z_i \in Z} \sum_{z_j \in Z} R_{f,m'}^{z_i}(t) T_{f,m}^{z_i,z_j} R_{f,m}^{z_j}(t) \tag{9}$$

Upon arrival of a service request at the server, the server's controller schedules and allocates the request stream to all corresponding microservice instances deployed on the server for processing. Based on Jackson's theory, these multiple instances can be modeled as an M/M/c queue model for request scheduling [72]. The M/M/c queue system features multiple microservice instances providing services in parallel. When a user request arrives, it can be immediately accepted by an idle instance for service. If no idle instances are available, the request enters a queue to wait until a microservice instance becomes available to process it. Therefore, the time consumed for processing can be divided into queuing and processing delay.

In practical scenarios, different service requests may utilize the same basic microservice. According to the superposition principle of Poisson processes, multiple independent Poisson input flows converging into the same queuing system will result in an output flow that is still a Poisson process, with a rate equal to the sum of the individual input rates. Therefore, the total arrival rate of requests for microservice m on edge server z at time t is defined as $\lambda_m^z(t)$. For any $m \in M$, it is given by:

$$\lambda_m^z(t) = \sum_{f \in F} R_{f,m}^z(t) \lambda_f(t) \tag{10}$$

Service intensity can describe system load and is an important basis for queue delay modeling. The service intensity of microservice m on server z at time t can be expressed as:

$$\rho_m^z(t) = \frac{\lambda_m^z(t)}{N_m^z(t) \mu_m^z(t)} \tag{11}$$

the service intensity $\rho_m^z(t)$ of a microservice instance must not exceed 1, otherwise it would lead to an infinitely long queue. Therefore, the following constraint must be satisfied:

$$\rho_m^z(t) < 1, \forall z \in Z, \forall m \in M \tag{12}$$

When the queue reaches steady state, average queuing delay for service requests in queue system at time t is given by:

$$T_{m,z}^q(t) = \frac{\rho_m^z(t) (\lambda_m^z(t))^{N_m^z(t)-1}}{N_m^z(t)! \left(1 - \rho_m^z(t)\right)^2 \left(\mu_m^z(t)\right)^{N_m^z(t)}} P_{0_m}^z(t) \tag{13}$$

The term $P_{0_m}^z$ is a normalization factor, whose physical meaning is the probability that the queue system is in an idle state. It is specifically represented as:

$$P_{0_m}^z(t) = \left(\sum_{j=0}^{N_m^z(t)-1} \frac{(j \rho_m^z(t))^j}{j!} + \frac{(N_m^z(t) \rho_m^z(t))^{N_m^z(t)}}{N_m^z(t)! \left(1 - \rho_m^z(t)\right)} \right)^{-1} \tag{14}$$

Therefore, at time t, the average queuing delay for all paths in request f is:

$$T_f^q(t) = \sum_{m \in M_f} \sum_{z \in Z} T_{m,z}^q(t) R_{f,m}^z(t) \tag{15}$$

Based on queuing network theory, the processing delay for microservice m being processed is:

$$T_{m,z}^c(t) = \frac{1}{\mu_m^z(t)} \tag{16}$$

Thus, the sum of the average processing delays for all microservices in request f is:

$$T_f^c(t) = \sum_{m \in M_f} \sum_{z \in Z} T_{m,z}^c(t) R_{f,m}^z(t) \tag{17}$$

In summary, the end-to-end response delay of service request f at time t can be divided into transmission delay, queuing delay, and processing delay:

$$T_f(t) = T_f^t(t) + T_f^q(t) + T_f^c(t) \tag{18}$$

Considering the requirement for service timeliness, the response time for each request must be less than its maximum tolerable delay:

$$T_f \le d_f^{\max}, \forall f \in F \tag{19}$$

Subject to the maximum tolerable delay constraint, one of our study's optimization goals is to minimize the average microservice processing delay. Considering the differences in request types and quantities at different times, we measure service quality as the average proportion of service delay relative to the maximum tolerable delay:

$$T(t) = \frac{1}{|F(t)|} \sum_{f \in F(t)} \frac{T_f(t)}{d_f^{\max}} \tag{20}$$

2) System Energy Consumption: Unlike cloud computing centers, edge computing systems have limited resources, making energy consumption a critical issue for the practical implementation of edge computing. In an edge system, the servers at edge servers are the primary sources of energy consumption [73]. The CPU, as a core component of the server, typically consumes the most energy during server operation. Therefore, this paper assumes that the power consumption of edge servers depends on the CPU utilization rate.

While higher operating frequencies can indeed increase service rates, they also lead to increased dynamic processing energy consumption and higher energy costs. Therefore, in dynamic scenarios, the server energy consumption model needs further refinement to reflect the impact of DVFS.

The power consumption of edge servers is mainly composed of static and dynamic power consumption. Static power consumption refers to the energy required to maintain the server in

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on September 16,2026 at 05:06:15 UTC from IEEE Xplore. Restrictions apply.

a stable operating state, which is related to parameters such as CPU frequency and voltage. This paper denotes it as P_z^{idle}. Dynamic power consumption comes from the processor executing tasks, caused by the switching actions of transistors. According to CMOS circuit theory, dynamic power consumption P_{dyn} is directly proportional to the square of operating frequency f and voltage V:

$$P_{\text{dyn}} \propto V^2 f \tag{21}$$

Since voltage V is approximately linearly related to frequency f, dynamic power consumption is proportional to the frequency cube, i.e., $P_{\text{dyn}} \propto f^3$. In contrast, static power consumption mainly comes from the leakage current in the circuit and is essentially unaffected by frequency changes.

Additionally, considering that the power consumption of a server is closely related to its load status, the CPU utilization rate $U_z(t)$ is introduced to represent the server's load level. Assuming the total CPU resource capacity of server z is CPU_z, and the total number of microservice instances deployed on server z at time t is $\sum_{m\in M} N_m^z(t)$, then the CPU utilization rate can be expressed as $U_z(t) = \frac{\sum_{m\in M} N_m^z(t)}{C_z}$.

Considering both dynamic and static power consumption, the power consumption of server z at time t can be expressed as:

$$P_z(t) = KU_z(t)f_z(t)^3 + P_z^{\text{idle}} \tag{22}$$

where K represents the power consumption coefficient determined by the fixed physical parameters of the server.

This paper adjusts the number of microservice instances and the frequency of edge servers dynamically to reduce the total energy consumption of the system. We take the proportion of system energy consumption to the maximum energy consumption of the system as the optimization objective:

$$E(t) = \frac{\sum_{z\in Z} P_z(t)}{\sum_{z\in Z} P_z^{\max}} \tag{23}$$

where $P_z^{\max}$ represents the power consumption of edge server z at maximum voltage.

Considering both delay and energy consumption, the optimization problem can be formulated as the following multi-objective optimization problem:

$$\min \sum_{t\in T} (w_1 T(t) + w_2 E(t)) \tag{24}$$

where w_1, w_2 are weight coefficients, and $w_1 + w_2 = 1$.

To ensure the feasibility and effectiveness of the optimization solution, the following constraints must also be considered:

$$T_f(t) < d_f^{\max}, \forall f \in F, t \in T \tag{25}$$

$$\sum_{m\in M} N_m^z(t) \le C_z, \forall z \in Z, t \in T \tag{26}$$

$$\rho_m^z(t) = \frac{\lambda_m^z(t)}{\mu_m^z(t)} < 1, \forall m \in M, z \in Z, t \in T \tag{27}$$

$$\sum_{z\in Z} R_{f,m}^z(t) = 1, \forall f \in F, m \in M, t \in T \tag{28}$$

$$R_{f,m}^z(t) > 0 \Rightarrow N_m^z(t) \ge 1, \forall z \in Z, t \in T \tag{29}$$

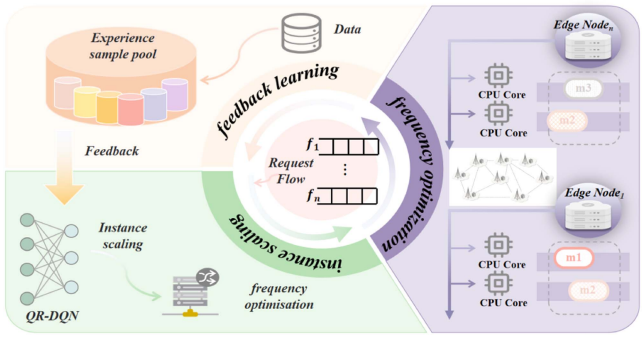


Fig. 2. Schematic Diagram of the RLDA Algorithm.

$$\sum_{k=1}^{K} x_{z,k}(t) = 1, \forall z \in Z, t \in T \tag{30}$$

Here, (25) indicates that the end-to-end delay for all requests must be less than their maximum tolerable delay to avoid service timeouts. (26) states that the total CPU resources of a server within each time slot should not exceed its maximum capacity. (27) ensures that the service intensity of each microservice instance within each time slot is less than 1, preventing infinite queue growth. (28) requires that the sum of request routing probabilities within each time slot equals 1, ensuring that all requests are routed to servers for processing. (29) specifies that requests can only be routed to servers where the corresponding microservices are deployed. (30) ensures that within the same time slot, each server can only select one performance state.

IV. Algorithm

This paper proposes a microservice deployment algorithm in dynamic edge computing environments, namely the RLDA. This algorithm utilizes reinforcement learning to dynamically determine the autoscaling of microservice instances, while integrating DVFS technology to optimize the operating frequency of servers.

The overall framework of the RLDA is shown in Fig. 2, which consists of two modules: the microservice autoscaling decision module based on reinforcement learning and the frequency adjustment module based on DVFS. The algorithm operates as follows: First, the reinforcement learning module generates an autoscaling plan for microservices according to request traffic and system state in the current time slot. Subsequently, the frequency adjustment module calculates the tolerable delay for each instance based on the deployment results and employs a greedy strategy to optimize the operating frequency of edge servers, thereby achieving the optimal balance between energy consumption and performance. Finally, the optimized performance metrics are fed back to the reinforcement learning module as rewards, driving the agent to learn and optimize the deployment strategy continuously. Through the three-stage closed-loop mechanism of instance scaling, frequency optimization, and feedback learning, the RLDA algorithm can adapt to complex and dynamic scenarios.

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on September 16,2026 at 05:06:15 UTC from IEEE Xplore. Restrictions apply.

A. The Autoscaling Modules Based on Reinforcement Learning

Reinforcement learning can solve decision-making problems by enabling the agent to interact with an unknown environment. The agent observes the current state $s \in S$ of the environment and executes an action $a \in A$ based on its parameterized policy $\pi_\theta(s)$. Subsequently, the agent receives an immediate reward $r \in R$ from the reward function, where the immediate reward represents the benefit obtained from action a. During the strategy training process, the agent aims to optimize its policy π_θ to maximize the cumulative expected reward function. Based on the above principles, this section defines the core elements of reinforcement learning, including the state space, action space, reward function, network architecture, and training process.

1) State Space: The state space of the microservice deployment problem primarily refers to the request arrival state, the current deployment state of the microservice instances, and the resource consumption state of the edge servers, which can be represented as follows:

$$s = \{s_r, s_s, s_c, s_m\} \tag{31}$$

Here, s_r represents the request arrival state, describing the state of service requests received by the system, including the type, size, and allowable delay of the requests. To capture dynamic changes in requests, in addition to current request information, it also includes historical request information from previous n time slots. s_s represents the current deployment state of microservice instances, describing the information about the microservice instances already deployed on the edge servers. s_c represents the resource consumption state of the edge servers, mainly referring to CPU consumption, which can be divided into used resource space and remaining resource space. To ensure the stability of strategy training, the two resource consumption states are normalized, with each element divided by its corresponding maximum value, making their values range between [0,1].

Considering the complexity of the microservice autoscaling scenario, adjusting the scaling of all microservice instances at once may lead to a large action space, making training difficult to converge. Therefore, this paper adopts a sequential decision-making method to accelerate convergence by fundamentally transforming the complexity of the optimization landscape. Specifically, within a time slot, the agent does not simultaneously decide the scaling of all microservices, which would result in an exponentially large action space that hinders effective exploration, but instead makes adjustment decisions for each type of microservice one by one, thereby decomposing the high-dimensional joint problem into a series of tractable low-dimensional sub-tasks. To facilitate this structured process, an additional state s_m is added to the state space to represent the microservice m that needs adjustment at the current time, serving as a dynamic pointer that focuses the learning algorithm on a specific sub-problem and ensures that the reward signal is clearly attributed to the corresponding action rather than being diluted across simultaneous changes. This mechanism significantly enhances sample efficiency and stabilizes the training process by allowing the agent to learn optimal policies for individual services sequentially while implicitly accounting for the evolving system state caused by previous adjustments, ultimately leading to much faster convergence compared to traditional joint decision-making approaches.

2) Action Space: The action space defines the set of operations that the agent can take at each time step. Since each decision is made for a specific type of microservice m, the actions can be represented as a binary tuple:

$$a = \{\Delta n, z\} \tag{32}$$

Here, Δn represents the adjustment value for the number of microservice instances. A positive value indicates an increase in the number of instances, while a negative value indicates a decrease. z represents the target server for the adjustment of the microservice instance count.

3) Reward Space: Reward functions are used to evaluate the quality of the agent's actions and guide the agent to learn the desired behavior. In this article, the goal of the algorithm is to reduce the latency and energy consumption of service requests. Therefore, the reward function is designed as the sum of the weights of the opposite numbers. The closer the reward is to 0, the better the optimization effect of the action on the system performance.

Considering that the action space in this paper is designed to adjust only one microservice instance at a time, while the overall latency and system energy consumption of service requests can only be accurately calculated after all microservices have been deployed. If the reward is calculated immediately after each microservice instance adjustment, it may lead to inaccurate rewards due to the adjustment of subsequent microservice instances. Therefore, the reinforcement learning objective of the RLDA algorithm is not to maximize the immediate reward of each action, but to maximize the final reward obtained after each deployment, guiding the agent to learn the globally optimal deployment strategy. To achieve this goal, a reward shaping and segment-based reward method is adopted to improve the reward.

Firstly, reward shaping is used to limit the reward space, ensuring that the reward value is within a reasonable range. By segmenting the reward to limit the value between $[R_{\min}, R_{\max}]$, the impact of extreme rewards on training can be avoided while maintaining the balance and stability of the reward. This reward shaping method can prevent the agent from being disturbed by excessively high or low reward values, thus better exploring and learning various deployment strategies. Let r_t represent the reward obtained at the end of the t-th time slot, then the reward shaping strategy can be expressed as follows:

$$r_t = \begin{cases} R_{\max} & \text{if } R_{\max} \le r_t \\ r_t & \text{if } R_{\min} < r_t < R_{\max} \\ R_{\min} & \text{if } r_t \le R_{\min} \end{cases} \tag{33}$$

Secondly, if rewards are only given at the end of a time slot, it can lead to sparse rewards, making it difficult for the agent to accurately judge the causal relationship between each action and the final reward. This increases the uncertainty of learning and the cost of trial and error. To overcome these issues, this paper sets up a segmented reward mechanism. The total system

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on September 16,2026 at 05:06:15 UTC from IEEE Xplore. Restrictions apply.

reward is calculated when all microservices have been deployed. At the same time, a small reward is given to the agent for each successful deployment. Conversely, a penalty is given for failure. This mechanism allows the agent to obtain valuable information from each deployment, thereby continuously optimizing its decision-making process.

4) Network Architectures: The algorithm employs an improved Quantile Regression Deep Q-Network (QR-DQN) for learning [74]. Traditional DQN algorithms aim to learn the expected value of Q-values, ignoring the distribution characteristics of Q-values, which may lead to estimation errors. Quantiles, which are key features of the distribution (such as the median, which is the 50th percentile), are used to describe the distribution of Q-values. Therefore, QR-DQN, based on the dual Q-network structure of the DQN, learns multiple quantile values to describe the distribution of Q-values, enabling it to make more robust and reliable decisions in complex environments.

QR-DQN uses quantile loss for network training. The goal of the loss function is to minimize the error between the predicted quantiles and the target quantiles. The loss function can be expressed as:

$$L(\tau) = \frac{1}{N} \sum_{i \in N} \sum_{j \in K} \rho_{\tau_j}(Q_{\tau_j}(s,a) - y_i) \tag{34}$$

Here, ρ_{τ_j} represents the loss for each quantile, and K is the number of quantiles.

To capture the temporal dependencies of request traffic, we introduce an LSTM network into the input layer of the Q-network. LSTM can process sequential data and learn the long-term dependencies within it. In this algorithm, the LSTM network takes the request arrival state S_t^r as input and outputs a hidden state vector, which encodes the temporal characteristics of the request traffic. At the same time, other state information is still processed by the fully connected layer.

To enhance the agent's exploration capability and cope with the uncertainty in dynamic environments, this paper introduces a Gaussian noise network. Unlike the traditional $\varepsilon-$greedy exploration strategy, the Gaussian noise network introduces noise into the parameter layer of the neural network, thus achieving more structured and adaptive exploration. Specifically, the fully connected layer in the online Q-network is replaced by a "noise linear layer." In the noise linear layer, the weights w and biases b are no longer fixed values but are modeled as learnable means (μ) and variances (σ), and combined with independent Gaussian noise (ϵ):

$$w = \mu_w + \sigma_w \odot \epsilon_w, \quad b = \mu_b + \sigma_b \odot \epsilon_b \tag{35}$$

Here, ϵ is independent Gaussian noise, $\odot$ represents element-wise multiplication. The target network still maintains a noise-free structure to ensure the stability of Q-value estimation.

5) Training Procedure: The proposed reinforcement learning module framework is shown in Fig. 3, and the overall process is outlined in Algorithm 1. Initially, the experience replay pool and neural network parameters are optimized. Upon receiving a request, the agent selects actions based on the policy to deploy microservices and obtain the next state. After all microservices are deployed, frequency adjustment is performed, and the reward is shared with the previous action. The deployment experience is stored in the experience replay pool. Subsequently, during the training phase, the agent randomly samples a batch of experience samples from the experience replay pool, updates the online Q-network parameters based on the loss function, and simultaneously updates the target Q-network parameters.

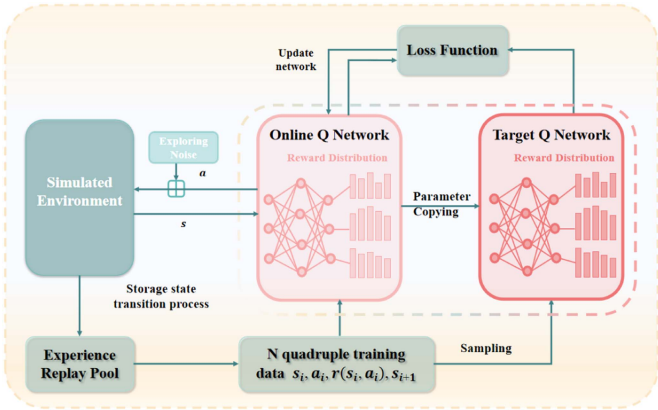


Fig. 3. An Improved Framework of the Quantile Regression Deep Q-Network Algorithm.

B. Frequency Adjustment Algorithm Based on DVFS

Building upon the microservice deployment plans obtained through reinforcement learning, this section further proposes a dynamic frequency adjustment algorithm based on DVFS. The algorithm aims to allocate the maximum tolerable delay for each microservice instance based on the determined deployment plan; then, under this constraint, it adjusts the frequency using a greedy algorithm to further reduce system energy consumption and latency.

1) Instance Latency Calculation Based on Deployment Plan: In edge computing scenarios, a single service request typically involves the collaborative processing of multiple microservices. However, due to the distributed nature of edge computing resources, these microservice instances may be deployed on different edge servers, causing the service request to pass through multiple different routing paths during execution. Additionally, compared to cloud computing centers, the communication latency across edge servers is higher and cannot be ignored. Therefore, in edge computing scenarios, the tolerable latency of instances of the same type of microservice may also vary. To enhance system performance and avoid excessive resource allocation, it is necessary to calculate this at the instance level.

Based on the aforementioned issues, this section presents an instance-level latency algorithm guided by the deployment plan. The core idea is to derive the maximum tolerable processing delay for each microservice instance, given the predetermined deployment configuration and request routing strategy. The pseudocode of the algorithm is shown in Algorithm 2.

The algorithm first iterates through all service requests, evenly distributing the end-to-end total latency of each request to the $|M_f|$ microservices on the path, obtaining the evenly distributed

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on September 16,2026 at 05:06:15 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: Improved Quantile Regression Deep Q-Network.

Input: Iteration count E, Online Q-Network Q, Target Q-Network Q', Training Frequency C;

Randomly initialize online Q-network $Q_\sigma(s,a)$ parameters σ and set target network parameters $\sigma' \leftarrow \sigma$;
Initialize experience replay buffer M_{replay};
for e in E **do**
  for t in T **do**
    for m in M **do**
      Retrieve current environmental state s_i;
      Select action a_i based on the online Q-network;
      Execute the action, transitioning the environmental state to s_{i+1};
      Perform frequency adjustment and calculate the reward $r(s_i, a_i)$;
    end for
    Insert the current experience data into the experience replay buffer;
  end for
  if $e\%C = 0$ **then**
    Sample N quadruples $\{s_i, a_i, r(s_i, a_i), s_{i+1}\}$ from the experience replay buffer;
    Use online Q-network to select actions and compute target Q-values with the target network;
    Compute the quantile regression loss;
    Update the online Q-network parameters σ using backpropagation;
    Update the target network parameters σ';
  end if
end for

Algorithm 2: Instance Latency Calculation Based on Deployment Scheme.

Input: Service request set F, microservice deployment plan D;

Output: Maximum tolerable delay $d_{m,z}^{\max}(t)$ for each microservice instance;

$d_{m,z}^{\max}(t) \leftarrow +\infty$;
for f in F **do**
  Calculate the average tolerable delay:
    $T_f^{per}(t) \leftarrow d_f^{\max}/|M_f|$;
  for m_i in M_f **do**
    Calculate the set of servers z_m hosting microservice m_i;
    for z in z_m **do**
      Determine the routing path for subsequent microservices using the routing strategy;
      Calculate the maximum transmission delay $T_{m,z}^{f}(t)$;
      Compute the tolerable delay for the microservice instance $d_{m,z}^{f}(t) = T_f^{per}(t) - T_{m,z}^{f}(t)$;
      if $d_{m,z}^{\max}(t) > d_{m,z}^{f}(t)$ **then**
        $d_{m,z}^{\max}(t) \leftarrow d_{m,z}^{f}(t)$;
      end if
    end for
  end for
end for
return $d_{m,z}^{\max}(t)$

latency. Then, for each specific microservice instance, it calculates the routing path and transmission latency to subsequent microservices based on the network topology and link status. The evenly distributed latency is then reduced by the maximum transmission latency to derive the tolerable processing latency for that microservice instance. Suppose that a microservice instance appears in multiple requests. In that case, the smallest latency threshold among them is taken as the tolerable latency for that instance $d_{m,z}^{max}(t)$ to avoid service timeouts and ensure a good user experience.

2) Latency-Constrained Greedy Frequency Scaling: Under the condition where the microservice deployment plan has been determined, the transmission delay of service requests and the static energy consumption of servers can be considered as constants. Therefore, the optimization goal depends only on the queuing delay, processing delay, and dynamic energy consumption. Since the frequency adjustments at edge servers are independent and the optimization objective function has a certain degree of decomposability, a greedy optimization strategy is adopted in this section to perform local optimization for each edge server. For edge server z, the local optimization objective function is defined as:

$$\min w_1 \frac{1}{|M_z(t)|} \sum_{m \in M_z(t)} \frac{(T_{m,z}^{q}(t) + T_{m,z}^{c}(t))}{d_{m,z}^{\max}(t)} + w_2 \frac{P_{z,\text{dyn}}(t)}{P_{z,\max}} \tag{36}$$

Here, $M_z(t)$ denotes the set of microservices currently deployed on edge node z. The pseudocode of the algorithm is shown in Algorithm 3.

V. Performance Evaluation

In this section, we evaluate the performance of the proposed microservice deployment algorithm for dynamic scenarios and compares it with benchmark algorithms under different traffic patterns. All experiments are conducted on a server equipped with an Intel Core i9-13900KF 3.00 GHz CPU, an RTX 4080 GPU, and a Windows 11 operating system. The algorithm implementation and agent network training are both based on the PyTorch deep learning framework.

A. Experimental Parameter Setting

To guarantee both the reliability and comparability of the experimental results, this study establishes a rigorously controlled experimental scenario by integrating the established findings in edge computing research [75], [76] with publicly available benchmark datasets. The specific experimental setup is described as follows:

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on September 16,2026 at 05:06:15 UTC from IEEE Xplore. Restrictions apply.

Algorithm 3: Latency-Constrained Greedy Frequency Scaling.

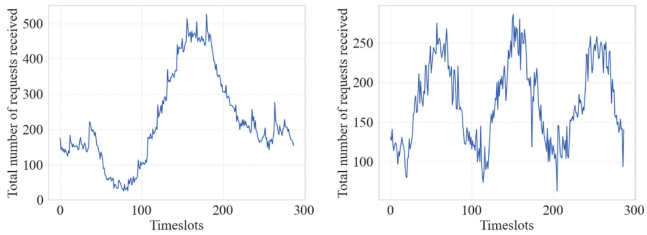
Input: The set of deployed microservice instances $M_z(t)$ for each edge node z, the maximum tolerable delay $d_{m,z}^{\max}(t)$ for each microservice instance m, the sorted set of available CPU frequency levels F_z for node z in ascending order;

Output: The optimal operating frequency f_z for each edge node z;

$min_cost \leftarrow cost$;
for z in Z **do**
  for m in $M_z(t)$ **do**
    Obtain $T_{m,z}^{q}(t)$ from (13);
    Obtain $T_{m,z}^{c}(t)$ from (16);
    Substitute into latency constraint:
    $T_{m,z}^{q}(t) + T_{m,z}^{c}(t) < d_{m,z}^{\max}(t)$;
    Solve for the minimum frequency $f_m^{\min}$ required by microservice m;
  end for
  $f_z^{\min} \leftarrow \max\{f_m^{\min} \mid m \in M_z(t)\}$
  $f_z \leftarrow f_z^{\min}$
  $min_cost \leftarrow \infty$
  for f in F_z and $f \geq f_z^{\min}$ **do**
    for m in $M_z(t)$ **do**
      Calculate $T_{m,z}^{c}(t)$;
    end for
    Calculate the local optimization objective value:
    $cost \leftarrow w_1 \frac{1}{|\mathbf{M}_z(t)|} \sum_{m \in \mathbf{M}_z(t)} \frac{(T_{m,z}^{q}(t)+T_{m,z}^{p}(t))}{d_m^{\max}(t)} + w_2 \frac{P_{z,max}(t)}{P_{z,max}}$;
    if $cost < min_cost$ **then**
      $f_z \leftarrow f$;
    end if
  end for
end for
return f_z

1) Physical Networks: To simulate the geographical distribution and coverage of services in a real edge computing environment, this experiment is referred to the dataset from the Shanghai Telecom base station [77]. Ten representative servers are selected from this dataset to build the edge service network. All edge servers incorporate DVFS technology and provide five discrete frequency levels: 0.8, 1.0, 1.2, 1.4, and 1.6 GHz, with 1 GHz as the baseline. To capture the heterogeneity of computational resources, each server is configured with 4–8 CPU cores. The network connections between servers are described using a bandwidth matrix, where each element represents the bandwidth value between any two servers, ranging from [500, 1000] Mbps, to reflect the differences in various links.

2) Microservices: This paper extracts information on microservice invocation and performance indicators from the Alibaba public dataset [78], and selects 20 microservices with different characteristics to simulate the diversity of microservices in real scenarios. The average service rate μ for each microservice is set within the range [20, 40] requests per minute, and its actual processing capacity follows an exponential distribution with mean μ, to simulate the performance variability inherent in real-world service instances.



(a) Twitter Traffic Pattern (b) FIFA World Cup Traffic Pattern

Fig. 4. Request Traffic Pattern.

3) User Requests: Based on the settings of the microservice parameters and request data statistics [2], each application request is modeled as a workflow involving 3 to 7 collaborating microservices. Inter-service data exchange is captured by assigning transmission volumes between collaborators uniformly at random within [0, 1000]. To reflect real-world load dynamics, we further design two representative traffic models that exhibit typical request fluctuation patterns, enabling a thorough evaluation of algorithm performance.

Twitter Traffic Model: As a global social media platform, Twitter (now known as X) exhibits complex request traffic characteristics typical of popular applications. Macroscopically, traffic shows significant temporal correlation, reflecting user activity patterns. On the other hand, sudden news events and trending topics can lead to strong burstiness and randomness in traffic for short periods. Therefore, this paper selects Twitter's application traffic data as the experimental benchmark.

World Cup Traffic Model: This paper uses the request volume data from a World Cup website over three months in 1998 as the research object. Analysis shows that this data exhibits highly similar daily patterns. To further verify the applicability and effectiveness of the proposed algorithm in periodic traffic scenarios, this paper selects traffic records from May 5th to May 8th as the experimental sample for the corresponding experimental studies.

To ensure the feasibility and validity of the experimental results, we conduct data sampling and scaling based on the load capacity and configuration of the experimental environment. Fig. 4 illustrates the changes in request arrival rates under the two pre-processed traffic models.

4) Hyperparameters: In the proposed RLDA, reinforcement learning employs the following parameters: a learning rate of 0.0001, a discount factor of 0.99, a replay buffer size of 10,000, and a batch size of 64 tuples drawn from the replay buffer.

B. Baselines

To fully verify the performance advantages of the proposed RLDA algorithm, several representative algorithms are selected as benchmarks for comparison:

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on September 16,2026 at 05:06:15 UTC from IEEE Xplore. Restrictions apply.

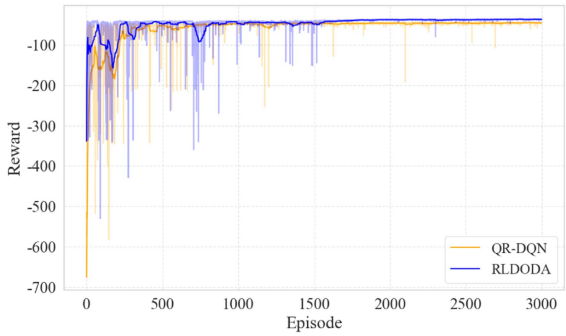
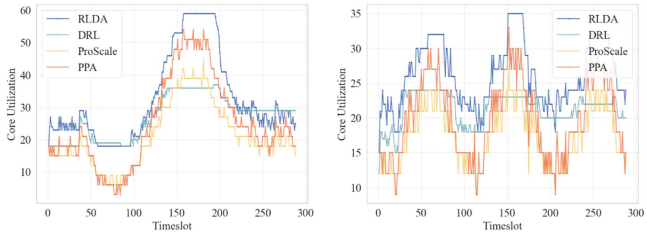


Fig. 5. Reward Convergence Curve.



(a) Twitter Traffic Pattern (b) FIFA World Cup Traffic Pattern

Fig. 6. Variations in Algorithm Core Utilization under Different Traffic Patterns.

DRL [79]: This algorithm uses independent hot coding and deep reinforcement learning to solve the dynamic deployment problem of microservices in mobile edge computing environments. According to the original experimental results, it has good performance in terms of resource utilization and service success rate. It represents the current DRL benchmark for addressing microservice deployment problems in edge environments.

ProScale [80]: To address dynamic workload fluctuations, the ProScale algorithm calculates the number of microservices required in real-time based on current workload and processing rate. It uses a proactive strategy to select servers during microservice deployment. It is the classic heuristic benchmark for addressing resource scheduling problems in dynamic load scenarios.

PPA [81]: PPA is an improved version of the classic Horizontal Pod Autoscaler (HPA) in the field of service scaling. Based on HPA, PPA comprehensively considers key indicators such as CPU utilization rate, job queue length, and request rate. Through prediction algorithms, PPA can preemptively determine whether indicators will exceed preset thresholds, thereby triggering the scaling operation of microservices promptly. In the realm of microservice autoscaling subject to multi-metric constraints, this approach stands as the prevailing optimized baseline.

To ensure the fairness of the experimental comparison, all comparison algorithms employ the on-demand frequency scaling strategy (OD) for frequency optimization. As one of the default governors in the Linux kernel, OD continuously adjusts the processor voltage and frequency in real time according to instantaneous CPU utilization.

C. Results

1) Reward Convergence Curve: As illustrated in Fig. 5, the average reward convergence curve of the RLDA algorithm during the training process demonstrates convergence of outcomes over time. To verify the effectiveness of the improvements made to the QR-DQN in this paper, the reinforcement learning module of the RLDA algorithm is replaced with the original QR-DQN algorithm for comparative experiments.

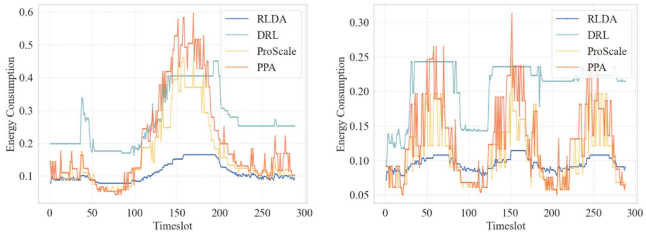
As the number of training rounds increases, both reward values exhibit a clear upward trend and gradually stabilize, indicating that the algorithm framework proposed in this paper can effectively learn an optimized microservice deployment strategy. In comparison, the RLDA demonstrates a more precipitous reward increase curve in the early stages of training, primarily due to the introduction of a noise network that enhances the diversity and randomness of policy exploration. This mechanism effectively guides the agent to escape from local optima and rapidly discovers potential efficient strategies in the initial phase, although this intensified exploration behavior also leads to greater fluctuations in the curve. As training progresses, the parameters within the noise network are gradually adjusted and optimized, resulting in more stable exploration behavior by the agent. In conclusion, the reward curve of RLDA becomes smoother and ultimately outperforms QR-DQN, thereby fully demonstrating the convergence performance of the proposed algorithm.

To verify the adaptability and robustness of each algorithm in real-world complex traffic scenarios, this paper conducted a comparative evaluation of four algorithms based on two traffic patterns.

2) Algorithm Performance Comparison Under Different Traffic Patterns: The variation in the number of CPU cores can intuitively reflect the scaling of microservice instances under traffic fluctuation scenarios, thereby effectively measuring the actual capability of algorithms in resource scheduling. Fig. 6 shows the trend of core usage for each algorithm under two typical traffic patterns, with changes in core usage over time. From the figure, it can be observed that all algorithms can adjust the number of instances according to dynamic changes in traffic, demonstrating a certain level of adaptability. However, RLDA exhibits superior dynamic resource adjustment capabilities. Due to the introduction of LSTM within the reinforcement learning structure, RLDA can better capture traffic trends and perform instance scaling from a global optimization perspective, rather than merely reacting passively to the current time slot's load. Consequently, the RLDA can proactively upscale instances before peak-load periods and postpone resource release during troughs, simultaneously optimizing latency and energy consumption while mitigating thrashing caused by traffic fluctuations.

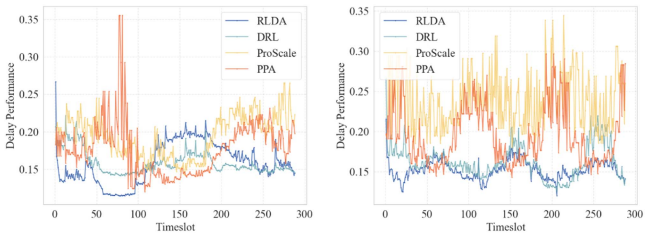
Fig. 7 illustrates the energy consumption metrics of the four algorithms under two traffic patterns over time. It can be observed that the algorithm proposed in this paper exhibits lower energy consumption and more stable curves with less fluctuation under both traffic patterns. This advantage is primarily attributed

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on September 16,2026 at 05:06:15 UTC from IEEE Xplore. Restrictions apply.



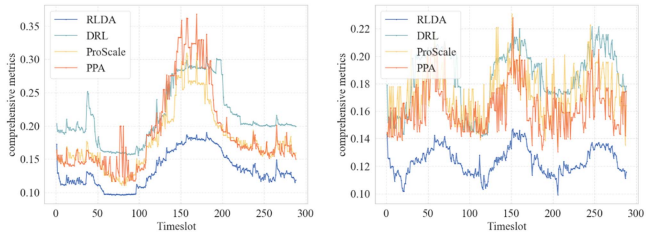
(a) Twitter Traffic Pattern (b) FIFA World Cup Traffic Pattern

Fig. 7. Energy Consumption Variations of Algorithm Systems under Different Traffic Patterns.



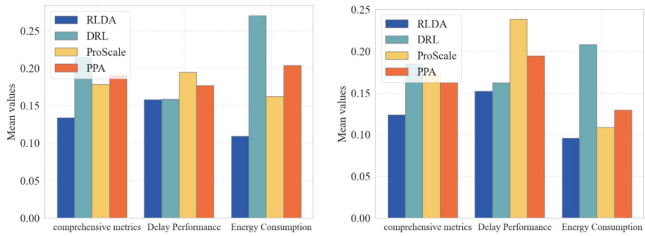
(a) Twitter Traffic Pattern (b) FIFA World Cup Traffic Pattern

Fig. 8. Variations in Algorithm Average Delay under Different Traffic Patterns.



(a) Twitter Traffic Pattern (b) FIFA World Cup Traffic Pattern

Fig. 9. Variations in Algorithm Comprehensive Performance Metrics under Different Traffic Patterns.



(a) Twitter Traffic Pattern (b) FIFA World Cup Traffic Pattern

Fig. 10. Mean Values of Algorithm Metrics under Different Traffic Patterns.

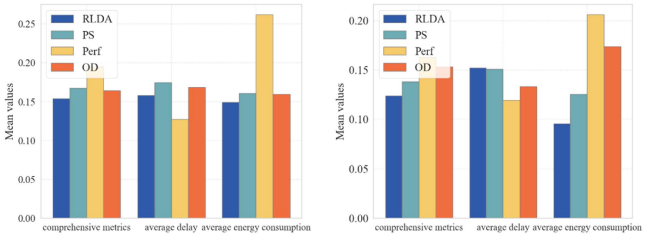
to the frequency adjustment strategy introduced in RLDA, which can finely and dynamically adjust the frequency of edge servers based on current traffic and optimization objectives. Consequently, even with a slightly higher number of cores, RLDA can effectively reduce overall energy consumption by controlling the operating frequency. This coordinated adjustment mechanism of core count and frequency fully demonstrates the advantages of the proposed algorithm in terms of energy consumption.

In edge computing scenarios, system response latency directly affects user experience and service quality, making latency control an important evaluation metric for algorithms. Fig. 8 displays the average latency of the four algorithms. While maintaining low energy consumption, RLDA still demonstrates good latency control capabilities. Its latency metric curve remains at a low level for most periods, with relatively small fluctuations. In contrast, the latency curves of ProScale and PPA algorithms exhibit larger fluctuations, as these two algorithms focus more on real-time request changes. Among them, ProScale's algorithm is relatively conservative when determining the number of instance scalings, leading to limited processing capability and thus more significant fluctuations in latency metrics. Fig. 9 shows the temporal trend of comprehensive metrics. It can be seen from the figure that RLDA consistently maintains the lowest comprehensive metric values throughout the entire operation period, and the curve changes smoothly, indicating that it achieves a better balance between energy consumption control and latency optimization. In comparison, other algorithms exhibit varying degrees of fluctuations in comprehensive performance, especially showing noticeable instability during peak load periods. This indicates that they struggle to balance multiple optimization objectives when facing dynamic loads, easily creating bottlenecks in certain dimensions, thereby increasing overall performance metrics.

To provide a more intuitive comparison of the overall performance of the four algorithms over the entire experimental period, Fig. 10 presents their average values across two dynamic traffic patterns in terms of comprehensive metrics, latency metrics, and energy consumption metrics. It can be observed from the figure that RLDA has the lowest average values for all three metrics in both scenarios, with the most significant advantages in the comprehensive and energy consumption metrics. Specifically, in the Twitter traffic pattern, the proposed algorithm achieves performance improvements of 37%, 25%, and 29% over DRL, ProScale, and PPA, respectively; in the World Cup traffic, the improvements are 33%, 28%, and 23%, respectively. These results fully validate the adaptability and generalization capability of the algorithm proposed in this paper in multi-objective scenarios. RLDA can effectively learn the load characteristics of different traffic patterns and employs joint optimization to coordinate instance scaling with frequency adjustment, achieving superior overall performance.

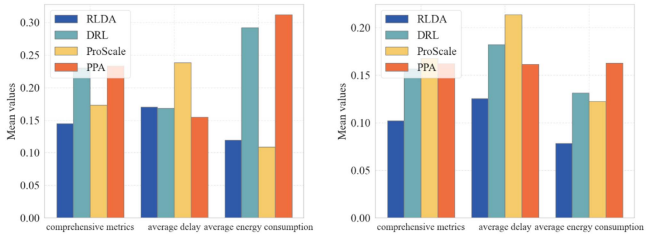
3) Impact of DVFS Strategy: The RLDA algorithm proposed in this paper integrates two mutually coupled modules: reinforcement learning and DVFS. To verify the effectiveness of the designed DVFS module, this study retains the reinforcement learning module in the experiments and compares it with three benchmark DVFS strategies. In addition to the previously mentioned on-demand frequency scaling strategy, these include the Performance (Perf) strategy, which prioritizes performance, and the PowerSaving (PS) strategy, which prioritizes energy saving. The Perf strategy maintains the processor at the highest voltage and frequency at all times, while the PS strategy maintains the lowest voltage.

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on September 16,2026 at 05:06:15 UTC from IEEE Xplore. Restrictions apply.



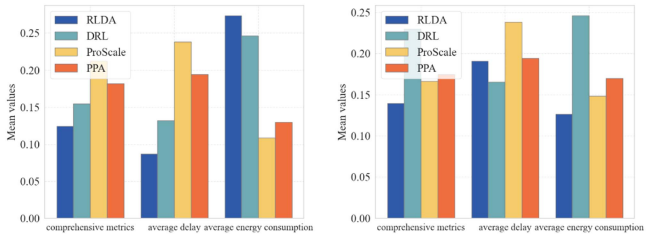
(a) Twitter Traffic Patter (b) FIFA World Cup Traffic Pattern

Fig. 11. The Impact of DVFS Policies.



(a) Small-Scale Edge System (b) Large-Scale Edge System

Fig. 12. The Impact of Edge System Scale.



(a) $w_1 = 0.8$, $w_2 = 0.2$ (b) $w_1 = 0.2$, $w_2 = 0.8$

Fig. 13. The Impact of Weight Coefficients.

Fig. 11 presents a comparison of the performance of various DVFS strategies under two different traffic patterns. The on-demand frequency scaling strategy is a system that adjusts the voltage based on the system load, thus demonstrating superior overall performance in comparison to strategies that invariably use a single frequency. However, it relies solely on the CPU utilization rate, lacking perception of the overall operational status of the microservice system. Consequently, its optimization effect is not as effective as that of the RLDA algorithm. The Perf strategy, while reducing processing latency, has been observed to result in significant increases in energy consumption. The PowerSaving strategy, due to the processor's low frequency, requires the deployment of more microservice instances to meet request processing needs, thus consuming more energy than RLDA. In contrast, the algorithm designed in this paper can achieve more refined energy control and performance optimization based on actual load and the needs of microservice applications, demonstrating significant advantages in all optimization objectives.

4) Impact of Edge System Scale: The edge system scale has a complex impact on algorithm performance. On one hand, system scale directly determines the amount of available resources; on the other hand, it significantly affects the complexity of the problem and the size of the optimization space. In small-scale edge systems, due to limited resources, issues such as service request timeouts and task queuing delays are likely to occur, posing high requirements for the scheduling algorithm's resource utilization efficiency and service assurance capabilities. In relatively resource-abundant large-scale edge systems, although a larger optimization space is provided, the expansion of the state space also increases the complexity of problem solving, challenging the algorithm's generalization ability and search efficiency.

Based on the aforementioned experimental settings, this paper constructs two experimental environments of different scales: one is a small-scale system with 7 edge servers, and the other is a large-scale system with 15 servers. Fig. 12 shows the performance metrics of the four algorithms under these two World Cup traffic scenarios. In the small-scale system, although RLDA is not optimal in terms of energy consumption and latency metrics, its comprehensive metric is significantly better than the other algorithms, achieving performance improvements of 49%, 19%, and 50% over DRL, ProScale, and PPA, respectively. This reflects its good resource scheduling and energy-saving capabilities. In the large-scale edge system, with the increase in resources and the expansion of the optimization space, algorithms based on RL can fully exert their advantage in strategy learning. RLDA achieves the best results in all three evaluation metrics, especially showing significant advantages in latency and energy consumption control. In comparison, DRL, also based on RL, performs better than ProScale and PPA in comprehensive metrics, but still significantly lags behind RLDA.

5) Impact of Weight Coefficients: To assess the impact of the weighting coefficients w_1 and w_2 in the composite objective function, we evaluate algorithm performance under two settings in the World C0up traffic scenario: $w_1 = 0.8$, $w_2 = 0.2$ and $w_1 = 0.2$, $w_2 = 0.8$. Fig. 13 reports the resulting composite, latency, and energy metrics.

As shown in Fig. 13, when $w_1 = 0.8$, $w_2 = 0.2$, the system optimization target is more focused on reducing request latency. Therefore, the algorithms tend to choose deployment and frequency adjustment strategies that result in lower latency. Both latency metrics and comprehensive metrics are relatively lower at this time. At this point, the latency metrics and comprehensive metrics of the RLDA are significantly better than those of the baseline algorithms. Instead, when $w_1 = 0.2$, $w_2 = 0.8$, the algorithms tend to choose strategies that consume less energy, but still take latency into account. The proposed algorithm can still achieve the best comprehensive average metrics. In summary, the selection of weighted coefficients should be balanced according to the specific needs of the actual application scenario. By adjusting the weighted coefficients, the RLDA can flexibly adapt to the preferences of different application scenarios for performance metrics, thereby achieving optimal performance optimization goals. In summary, RLDA achieves comprehensive optimization of latency and low energy consumption of various dynamic traffic patterns, demonstrating stronger adaptability and robustness compared to traditional deployment strategies.

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on September 16,2026 at 05:06:15 UTC from IEEE Xplore. Restrictions apply.

VI. Conclusion

This paper has addressed the autoscaling of microservice instances in response to varying service request loads in practical applications by focusing on the joint optimization of autoscaling and frequency adjustment under load fluctuations. We first established a fine-grained multi-instance service model to jointly optimize instance readjustment and request routing, and in the proposed RLDA framework, an enhanced quantile regression DQN was utilized to make decisions about microservice instance autoscaling in response to dynamic traffic changes. Subsequently, DVFS technology was integrated to further control the processor frequencies of edge servers, thereby achieving a trade-off between latency and energy consumption. Extensive simulation results demonstrate that under the Twitter traffic pattern, our algorithm achieves comprehensive performance improvements in energy consumption and latency of 37%, 25%, and 29% compared to DRL, ProScale, and PPA, respectively. Under the World Cup traffic pattern, the gains are 33%, 28%, and 23%, respectively. Furthermore, treating energy consumption as a core optimization objective aligns well with current sustainability trends, and this research holds significant societal value within the broader context of Green Edge Computing. However, we acknowledge that our current model assumes a relatively homogeneous node environment and linear service dependency structures, which may not fully capture the complexities of real-world heterogeneous edge infrastructures where devices vary significantly in computing power and energy profiles. Consequently, our future work will prioritize extending the framework to accommodate node heterogeneity and complex Directed Acyclic Graph service dependencies to ensure broader applicability across diverse hardware landscapes.

References

[1] X. Shang, Y. Mao, Y. Liu, Y. Huang, Z. Liu, and Y. Yang, "Online container scheduling for data-intensive applications in serverless edge computing," in *Proc. IEEE Conf. Comput. Commun.*, 2023, pp. 1–10.

[2] M. S. Aslanpour et al., "Serverless edge computing: Vision and challenges," in *Proc. Australas. Comput. Sci. Week Multiconf.*, 2021, pp. 1–10.

[3] X. Wang, Y. Han, V. C. M. Leung, D. Niyato, X. Yan, and X. Chen, "Convergence of edge computing and deep learning: A comprehensive survey," *IEEE Commun. Surv. Tut.*, vol. 22, no. 2, pp. 869–904, Second quarter 2020.

[4] Alibaba, "Alibaba cluster data," 2026. [Online]. Available: https://github.com/alibaba/clusterdata

[5] Microsoft, "Dapr," 2026. [Online]. Available: https://dapr.io

[6] A. Samanta, L. Jiao, M. Mühlhäuser, and L. Wang, "Incentivizing microservices for online resource sharing in edge clouds," in *Proc. IEEE 39th Int. Conf. Distrib. Comput. Syst.*, 2019, pp. 420–430.

[7] K. Peng et al., "Symmetric orchestration under service mesh paradigm: Empowering massive online applications in edge clouds," *IEEE Trans. Mobile Comput.*, vol. 25, no. 3, pp. 4300–4316, Mar. 2026.

[8] Z. Li and W. Bai, "Energy consumption minimization scheme with dynamic computational windows for UAV-assisted edge computing," in *Proc. 5th Int. Conf. Intell. Comput. Hum.- Comput. Interaction*, 2024, pp. 144–148.

[9] Y. Hu et al., "Time-varying microservice orchestration with routing for dynamic call graphs via multi-scale deep reinforcement learning," *IEEE Trans. Serv. Comput.*, vol. 18, no. 5, pp. 3276–3291, Sep./Oct. 2025.

[10] P. Mach and Z. Becvar, "Mobile edge computing: A survey on architecture and computation offloading," *IEEE Commun. Surv. Tut.*, vol. 19, no. 3, pp. 1628–1656, Third quarter 2017.

[11] S. Abrishami, M. Naghibzadeh, and D. H. Epema, "Deadline-constrained workflow scheduling algorithms for infrastructure as a service clouds," *Future Gener. Comput. Syst.*, vol. 29, no. 1, pp. 158–169, 2013.

[12] S. Guo, Y. Qi, Y. Jin, W. Li, X. Qiu, and L. Meng, "Endogenous trusted DRL-based service function chain orchestration for IoT," *IEEE Trans. Comput.*, vol. 71, no. 2, pp. 397–406, Feb. 2022.

[13] K. Peng et al., "Large-scale service mesh orchestration with probabilistic routing in cloud data centers," *IEEE Trans. Serv. Comput.*, vol. 18, no. 2, pp. 868–882, Mar./Apr. 2025.

[14] Y. Hu et al., "Collaborative orchestration with probabilistic routing for dynamic service mesh in clouds," in *Proc. IEEE Conf. Comput. Commun.*, 2025, pp. 1–10.

[15] G. Tong, C. Meng, S. Song, M. Pan, and Y. Yu, "GMA: Graph multi-agent microservice autoscaling algorithm in edge-cloud environment," in *Proc. IEEE Int. Conf. Web Serv.*, 2023, pp. 393–404.

[16] K. Cheng et al., "ProScale: Proactive autoscaling for microservice with time-varying workload at the edge," *IEEE Trans. Parallel Distrib. Syst.*, vol. 34, no. 4, pp. 1294–1312, Apr. 2023.

[17] M. Hu et al., "Joint optimization of microservice deployment and routing in edge via multi-objective deep reinforcement learning," *IEEE Trans. Netw. Service Manag.*, vol. 21, no. 6, pp. 6364–6381, Dec. 2024.

[18] Y. Chai, K. Gao, G. Zhang, L. Lu, Q. Li, and Y. Zhang, "Joint task offloading, resource allocation and model placement for ai as a service in 6G network," *IEEE Trans. Serv. Comput.*, vol. 17, no. 6, pp. 3830–3843, Nov./Dec. 2024.

[19] Y. Hu et al., "AI service deployment and resource allocation optimization based on human-like networking architecture," *IEEE Internet Things J.*, vol. 11, no. 14, pp. 24795–24813, Jul. 2024.

[20] Y. Li, X. Gao, M. Shi, J. Kang, D. Niyato, and K. Yang, "Dynamic weighted energy minimization for aerial edge computing networks," *IEEE Internet Things J.*, vol. 12, no. 1, pp. 683–697, Jan. 2025.

[21] Y. Hu et al., "Energy-aware service mesh deployment and online request routing in edge: A hierarchical deep reinforcement learning approach," *IEEE Trans. Mobile Comput.*, 2026.

[22] K. Fu, W. Zhang, Q. Chen, D. Zeng, and M. Guo, "Adaptive resource efficient microservice deployment in cloud-edge continuum," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 8, pp. 1825–1840, Aug. 2022.

[23] J. Bi, K. Zhang, H. Yuan, and J. Zhang, "Energy-efficient computation offloading for static and dynamic applications in hybrid mobile edge cloud system," *IEEE Trans. Sustain. Comput.*, vol. 8, no. 2, pp. 232–244, Apr.–Jun. 2023.

[24] B. Yang et al., "A joint energy and latency framework for transfer learning over 5G industrial edge networks," *IEEE Trans. Ind. Inform.*, vol. 18, no. 1, pp. 531–541, Jan. 2022.

[25] K. Peng, L. Wang, J. He, C. Cai, and M. Hu, "Joint optimization of service deployment and request routing for microservices in mobile edge computing," *IEEE Trans. Serv. Comput.*, vol. 17, no. 3, pp. 1016–1028, May/Jun. 2024.

[26] Y. Li, H. Zhang, W. Tian, and H. Ma, "Joint optimization of auto-scaling and adaptive service placement in edge computing," in *Proc. IEEE 27th Int. Conf. Parallel Distrib. Syst.*, 2021, pp. 923–930.

[27] E. Incerto, R. Pizziol, and M. Tribastone, "μOPT: An efficient optimal autoscaler for microservice applications," in *Proc. IEEE Int. Conf. Autonomic Comput. Self-Organizing Syst.*, 2023, pp. 67–76.

[28] I. Dimolitsas, D. Spatharakis, D. Dechouniotis, A. Zafeiropoulos, and S. Papavassiliou, "Multi-application hierarchical autoscaling for kubernetes edge clusters," in *Proc. IEEE Int. Conf. Smart Comput.*, 2023, pp. 291–296.

[29] A. A. Pramesti and A. I. Kistijantoro, "Autoscaling based on response time prediction for microservice application in kubernetes," in *Proc. 9th Int. Conf. Adv. Inform., Concepts, Theory Appl.*, 2022, pp. 1–6.

[30] L. M. Al Qassem, T. Stouraitis, E. Damiani, and I. A. M. Elfadel, "Proactive random-forest autoscaler for microservice resource allocation," *IEEE Access*, vol. 11, pp. 2570–2585, 2023.

[31] J. Li, S. Li, J. Tan, D. Jin, S. Chen, and J. Yang, "DCScaler: Spatiotemporal prediction aided distributed collaborative autoscaling of microservices," in *Proc. IEEE 10th Int. Conf. Edge Comput. Scalable Cloud*, 2024, pp. 42–47.

[32] Z. G. Chen, J.-W. Chang, C. Chen, C.-Y. Li, C.-C. Huang, and L.-C. Wang, "Horizontal pod autoscaling for precise startup of AI microservices at the network edge: A hybrid proactive and reactive approach," in *Proc. 33rd Wireless Opt. Commun. Conf.*, 2024, pp. 192–197.

[33] A. A. Khaleq and I. Ra, "Intelligent autoscaling of microservices in the cloud for real-time applications," *IEEE Access*, vol. 9, pp. 35464–35476, 2021.

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on September 16,2026 at 05:06:15 UTC from IEEE Xplore. Restrictions apply.

[34] H. Nayir, A. Demir, and H. Kaya, "Multi-dimensional autoscaling in microservices using reinforcement learning," in *Proc. 33rd Int. Symp. Model., Anal. Simul. Comput. Telecommun. Syst.*, 2025, pp. 1–8.

[35] N. Ding, P. Yu, F. Zhang, W. Huang, S. Guo, and F. Qi, "Joint autoscaling method for heterogeneous containers in power systems based on improved DRL algorithm," in *Proc. IEEE 7th Adv. Inf. Manage., Commun., Electron. Automat. Control Conf.*, 2025, pp. 1517–1524.

[36] K. Peng et al., "Global microservice autoscaling over heterogeneous edge environments for internet applications: A reinforcement learning approach," *IEEE Internet Things J.*, vol. 12, no. 19, pp. 40247–40258, Oct. 2025.

[37] C. Meng, S. Song, H. Tong, M. Pan, and Y. Yu, "DeepScaler: Holistic autoscaling for microservices based on spatiotemporal GNN with adaptive graph learning," in *Proc. 38th IEEE/ACM Int. Conf. Automated Softw. Eng.*, 2023, pp. 53–65.

[38] Z. Fang, H. Ma, G. Chen, and R. Buyya, "HGraphScale: Hierarchical graph learning for autoscaling microservice applications in container-based cloud computing," *IEEE Trans. Serv. Comput.*, vol. 19, no. 1, pp. 410–422, Jan./Feb. 2026.

[39] X. He, Z. Tu, M. Wagner, X. Xu, and Z. Wang, "Online deployment algorithms for microservice systems with complex dependencies," *IEEE Trans. Cloud Comput.*, vol. 11, no. 2, pp. 1746–1763, Apr.–Jun. 2023.

[40] H. Li, B. Tang, W. Xu, F. Guo, and X. Zhang, "Application deployment in mobile edge computing environment based on microservice chain," in *Proc. IEEE 25th Int. Conf. Comput. Supported Cooperative Work Des.*, 2022, pp. 531–536.

[41] W. Lv et al., "Microservice deployment in edge computing based on deep q learning," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 11, pp. 2968–2978, Nov. 2022.

[42] C. Hu, Z. Cheng, D. Wu, Y. Wang, F. Liu, and Z. Qiu, "Structural generalization for microservice routing using graph neural networks," in *Proc. 3rd Int. Conf. Artif. Intell. Automat. Control*, 2025, pp. 278–282.

[43] P. R. Desai, S. Mini, and D. K. Tosh, "Edge-based optimal routing in SDN-enabled industrial Internet of Things," *IEEE Internet Things J.*, vol. 9, no. 19, pp. 18898–18907, Oct. 2022.

[44] X. Liu et al., "Message passing DQN enhanced fault tolerance traffic routing for dynamic 6G edge networks," in *Proc. IEEE Netw. Operations Manage. Symp.*, 2025, pp. 1–8.

[45] B. He, J. Wang, Q. Qi, H. Sun, and J. Liao, "RTHop: Real-time hop-by-hop mobile network routing by decentralized learning with semantic attention," *IEEE Trans. Mobile Comput.*, vol. 22, no. 3, pp. 1731–1747, Mar. 2023.

[46] D. M. Casas-Velasco, O. M. C. Rendon, and N. L. S. da Fonseca, "DRSIR: A deep reinforcement learning approach for routing in software-defined networking," *IEEE Trans. Netw. Service Manag.*, vol. 19, no. 4, pp. 4807–4820, Dec. 2022.

[47] J. Liang, S. Huang, Y. Qiu, L. Liu, F. Aziz, and M. Chen, "Sustainable virtual network function placement and traffic routing for green mobile edge networks," *IEEE Trans. Green Commun. Netw.*, vol. 8, no. 4, pp. 1450–1465, Dec. 2024.

[48] P. Lai et al., "Intelligent autoscaling of microservice and request routing for dynamic service requests," *IEEE Internet Things J.*, vol. 12, no. 21, pp. 44468–44491, Nov. 2025.

[49] H. Xiong et al., "Joint optimization of communication-aware group microservice deployment and multi-chain request routing," *IEEE Trans. Serv. Comput.*, vol. 19, no. 2, pp. 1305–1320, Mar./Apr. 2026.

[50] K. Peng et al., "Delay-aware joint microservice deployment and request routing in multi-edge environments based on reinforcement learning," *IEEE Trans. Netw. Service Manag.*, vol. 22, no. 4, pp. 2971–2986, Aug. 2025.

[51] K. Peng et al., "Hybrid orchestration of AI services and microservices in cloud-edge collaboration," *IEEE Trans. Mobile Comput.*, 2026.

[52] S. Zhang et al., "Joint deployment and routing for hybrid AI services and microservices in edge via deep reinforcement learning," *IEEE Internet Things J.*, vol. 13, no. 7, pp. 13455–13471, Apr. 2026.

[53] K. Peng et al., "Collaborative orchestration of microservices and AI services in edges: A dual-time-scale reinforcement learning approach," *IEEE Trans. Serv. Comput.*, 2026.

[54] L. Gu, W. Zhang, Z. Wang, D. Zeng, and H. Jin, "Service management and energy scheduling toward low-carbon edge computing," *IEEE Trans. Sustain. Comput.*, vol. 8, no. 1, pp. 109–119, Jan.–Mar. 2023.

[55] M. Teng, X. Li, and K. Zhu, "Joint optimization of sequential task offloading and service deployment in end-edge-cloud system for energy efficiency," *IEEE Trans. Sustain. Comput.*, vol. 9, no. 3, pp. 283–298, May/Jun. 2024.

[56] W. Rao and H. Li, "Energy-aware scheduling algorithm for microservices in kubernetes clouds," *J. Grid Comput.*, vol. 23, no. 1, pp. 1–22, 2025.

[57] G. Premsankar and B. Ghaddar, "Energy-efficient service placement for latency-sensitive applications in edge computing," *IEEE Internet Things J.*, vol. 9, no. 18, pp. 17926–17937, Sep. 2022.

[58] R. Wang et al., "Joint energy and computation workload management for geo-distributed data centers," *IEEE Trans. Green Commun. Netw.*, vol. 9, no. 4, pp. 2115–2128, Dec. 2025.

[59] G. Sun, R. Zhou, J. Sun, H. Yu, and A. V. Vasilakos, "Energy-efficient provisioning for service function chains to support delay-sensitive applications in network function virtualization," *IEEE Internet Things J.*, vol. 7, no. 7, pp. 6116–6131, Jul. 2020.

[60] A. Varasteh, B. Madiwalar, A. Van Bemten, W. Kellerer, and C. Mas-Machuca, "Holu: Power-aware and delay-constrained VNF placement and chaining," *IEEE Trans. Netw. Service Manag.*, vol. 18, no. 2, pp. 1524–1539, Jun. 2021.

[61] Z. Zhang, Y. Zhao, H. Li, C. Lin, and J. Liu, "DVFO: Learning-based DVFS for energy-efficient edge-cloud collaborative inference," *IEEE Trans. Mobile Comput.*, vol. 23, no. 10, pp. 9042–9059, Oct. 2024.

[62] M. I. Khaleel, M. Safran, S. Alfarhood, and D. Gupta, "Combinatorial metaheuristic methods to optimize the scheduling of scientific workflows in green DVFS-enabled edge-cloud computing," *Alexandria Eng. J.*, vol. 86, pp. 458–470, 2024.

[63] L. Wang, X. Liu, H. Ding, Y. Hu, K. Peng, and M. Hu, "Energy-delay-aware joint microservice deployment and request routing with DVFS in edge: A reinforcement learning approach," *IEEE Trans. Comput.*, vol. 74, no. 5, pp. 1589–1604, May 2025.

[64] D. Liu, S.-G. Yang, Z. He, M. Zhao, and W. Liu, "CARTAD: Compiler-assisted reinforcement learning for thermal-aware task scheduling and DVFS on multicores," *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, vol. 41, no. 6, pp. 1813–1826, Jun. 2022.

[65] A. Yeganeh-Khaksar, M. Ansari, S. Safari, S. Yari-Karin, and A. Ejlali, "Ring-DVFS: Reliability-aware reinforcement learning-based DVFS for real-time embedded systems," *IEEE Embedded Syst. Lett.*, vol. 13, no. 3, pp. 146–149, Sep. 2021.

[66] F. Chen, H. Yu, W. Jiang, and Y. Ha, "Quality optimization of adaptive applications via deep reinforcement learning in energy harvesting edge devices," *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, vol. 41, no. 11, pp. 4873–4886, Nov. 2022.

[67] S. K. Panda, M. Lin, and T. Zhou, "Energy-efficient computation offloading with DVFS using deep reinforcement learning for time-critical IoT applications in edge computing," *IEEE Internet Things J.*, vol. 10, no. 8, pp. 6611–6621, Apr. 2023.

[68] Q. Zhang, M. Lin, L. T. Yang, Z. Chen, and P. Li, "Energy-efficient scheduling for real-time systems based on deep Q-learning model," *IEEE Trans. Sustain. Comput.*, vol. 4, no. 1, pp. 132–141, Jan.–Mar. 2019.

[69] Q. Zhang, M. Lin, L. T. Yang, Z. Chen, S. U. Khan, and P. Li, "A double deep Q-learning model for energy-efficient edge scheduling," *IEEE Trans. Serv. Comput.*, vol. 12, no. 5, pp. 739–749, Sep./Oct. 2019.

[70] H. Huang, M. Lin, L. T. Yang, and Q. Zhang, "Autonomous power management with double-Q reinforcement learning method," *IEEE Trans. Ind. Informat.*, vol. 16, no. 3, pp. 1938–1946, Mar. 2020.

[71] J. Zhang, X. Zhou, T. Ge, X. Wang, and T. Hwang, "Joint task scheduling and containerizing for efficient edge computing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 8, pp. 2086–2100, Aug. 2021.

[72] E. Gelenbe, G. Pujolle, E. Gelenbe, and G. Pujolle, in *Introduction to Queueing Networks*. Hoboken, NJ, USA: Wiley, 1987.

[73] C. Jiang et al., "Energy aware edge computing: A survey," *Comput. Commun.*, vol. 151, pp. 556–580, 2020.

[74] W. Dabney, M. Rowland, M. Bellemare, and R. Munos, "Distributional reinforcement learning with quantile regression," in *Proc. AAAI Conf. Artif. Intell.*, 2018, vol. 32, no. 1, pp. 2892–2901.

[75] X. Ma, A. Zhou, S. Zhang, and S. Wang, "Cooperative service caching and workload scheduling in mobile edge computing," in *Proc. IEEE Conf. Comput. Commun.*, 2020, pp. 2076–2085.

[76] Y. Bi, C. C. Meixner, M. Bunyakitanon, X. Vasilakos, R. Nejabati, and D. Simeonidou, "Multi-objective deep reinforcement learning assisted service function chains placement," *IEEE Trans. Netw. Service Manag.*, vol. 18, no. 4, pp. 4134–4150, Dec. 2021.

[77] S. Wang, Y. Zhao, J. Xu, J. Yuan, and C.-H. Hsu, "Edge server placement in mobile edge computing," *J. Parallel Distrib. Comput.*, vol. 127, pp. 160–168, 2019.

[78] C. Lu, W. Chen, K. Ye, and C.-Z. Xu, "Understanding the workload characteristics in alibaba: A view from directed acyclic graph analysis," in *Proc. Int. Conf. High Perform. Big Data Intell. Syst.*, 2020, pp. 1–8.

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on September 16,2026 at 05:06:15 UTC from IEEE Xplore. Restrictions apply.

[79] Y. Zhao, J. Wang, and B. Li, "A deep reinforcement learning approach to online microservice deployment in mobile edge computing," in *Proc. Int. Conf. Service-Oriented Comput.*, 2023, pp. 127–142.
[80] F. Guan, L. Peng, and J. Qiao, "A fluid scheduling algorithm for DAG tasks with constrained or arbitrary deadlines," *IEEE Trans. Comput.*, vol. 71, no. 8, pp. 1860–1873, Aug. 2022.
[81] L. Ju, P. Singh, and S. Toor, "Proactive autoscaling for edge computing systems with kubernetes," in *Proc. 14th IEEE/ACM Int. Conf. Utility Cloud Comput. Companion*, 2021, pp. 1–8.



Bo Xu is currently workig toward the Ph.D. degree with the School of Electronic Information and Communications, Huazhong University of Science and Technology, Wuhan, China. He is also the Deputy General Manager of Hubei Chutianyun Company, Ltd., Wuhan. His research interests include cloud computing and distributed systems.



Ruyao Zou is currently working toward the Ph.D. degree with the School of Electronic Information and Communications, Huazhong University of Science and Technology, Wuhan, China. Her research interests include edge computing and microservice deployment.



Jiangtian Nie received the Ph.D. degree from Interdisciplinary Graduate School, Nanyang Technological University, Singapore. She is currently a Lecturer with the Computer Science Department, University of Aberdeen, Aberdeen, U.K. Her research interests include edge intelligence, wireless communications, and Internet of Things.



Zhiheng Yao received the B.E. degree from the Wuhan University of Technology, Wuhan, China, in 2024. She is currently working toward the M.S. degree with the School of Electronic Information and Communication, Huazhong University of Science and Technology, Wuhan. Her research interests include reinforcement learning, microservice deployment, and migration.



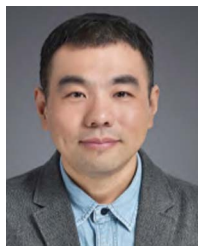
Yi Hu is currently working toward the Ph.D. degree with the School of Electronic Information and Communications, Huazhong University of Science and Technology, Wuhan, China. His research interests include cloud computing and distributed systems.



Hao Wen received the B.E. degree in measurement and control technology and instrumentation from Jilin University, Changchun, China, in 2022. He is currently working toward the master's degree in information and communication engineering with the School of Electronic Information and Communication, Huazhong University of Science and Technology, Wuhan. His research interests include IoT and edge computing.



Menglan Hu received the B.E. degree in electronic and information engineering from the Huazhong University of Science and Technology, Wuhan, China, in 2007, and the Ph.D. degree in electrical and computer engineering from the National University of Singapore, Singapore, in 2012. He is currently an Associate Professor with the School of Electronic Information and Communications, Huazhong University of Science and Technology. His research interests include cloud computing, computer networks, distributed systems, and mobile computing.



Kai Peng (Member, IEEE) received the B.E., M.S., and Ph.D. degrees from the Huazhong University of Science and Technology, Wuhan, China, in 1999, 2002, and 2006, respectively. He is currently a Professor with the School of Electronic Information and Communications, Huazhong University of Science and Technology. His research interests include cloud computing and computer networks.

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on September 16,2026 at 05:06:15 UTC from IEEE Xplore. Restrictions apply.